

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

A. File Structure Directory

1. Code Editor Frontend

```
InsureSafe_AI_Insurance_System
|
|— templates
|   |— base.html
|   |— index.html
|   |— dashboard.html
|   |— policies.html
|   |— profile.html
|   |— admin.html
|   |— admin_policy_docs.html
|   |— apply_policy.html
|   |— apply_claim.html
|   |— claims_ai.html
|   |— document_verification.html
|   |— face_verification.html
|   |— auth_login.html
|   |— auth_signup.html
|   |— auth_face.html
|   └─ auth_forgot.html
|
|— static
|   |— style.css
|   └─ js
|       └─ app.js
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

2. Code Editor Backend

InsureSafe_AI_Insurance_System

|

└─ app.py

└─ db.py

└─ custom_policies_unique_50.csv

└─ requirements.txt

|

└─ services

|

└─ claims_ai.py

|

└─ face_verify.py

|

└─ ocr_verify.py

|

└─ otp_service.py

|

└─ recommender.py

|

└─ vehicle_damage_ai.py

|

└─ Documentation

|

└─ Project_Report.pdf

|

└─ Architecture_Diagram.png

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
|  |— Flow_Diagram.png
|  └─ README.txt
```

B. Collaborative code Editor

1. app.py

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, abort,
jsonify
from flask_mail import Mail, Message
import os, time, random
from werkzeug.utils import secure_filename
from datetime import datetime, timedelta
from functools import wraps
from itsdangerous import URLSafeTimedSerializer
from bson.objectid import ObjectId
from werkzeug.exceptions import RequestEntityTooLarge
import base64
from io import BytesIO
from PIL import Image
from services import otp_service
from bson import ObjectId

# ---- Services ----
from services.recommender import recommend_policies
from services.face_verify import FaceVerifier
from services.ocr_verify import DocumentVerifier
from services.claims_ai import ClaimsAI
from services.otp_service import send_otp, verify_otp

# ---- Database Helpers ----
from db import (
    init_db, add_user, authenticate_user, get_user_by_id, get_user_by_email,
    add_policy, get_policy_by_id, assign_policy_to_user, get_user_policies,
    update_user_policy_status, add_claim, db_update_claim_status, toggle_user_active,
    get_all_users, get_all_policies, hash_password, users_col, claims_col, policies_col,
    user_policies_col
)

# =====
# APP CONFIG
# =====
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
app = Flask(__name__)
app.secret_key = os.getenv("SECRET_KEY", "dev-secret")
app.config["UPLOAD_FOLDER"] = "uploads"
app.config["ID_PHOTOS"] = "id_photos"
app.config["DEMO_MODE"] = os.getenv("DEMO_MODE", "false").lower() == "true"
app.config['MAX_CONTENT_LENGTH'] = 20 * 1024 * 1024

# ===== EMAIL (BREVO SMTP) =====
app.config["MAIL_SERVER"] = "smtp-relay.brevo.com"
app.config["MAIL_PORT"] = 587
app.config["MAIL_USE_TLS"] = True
app.config["MAIL_USERNAME"] = "a003a2001@smtp-brevo.com"
app.config["MAIL_PASSWORD"] = "xsmtpsib-
338ccb36a9bdc0db0df009780089229a11ec700eb5d672be1716386932f06a2a-
v7dHZxhYcXxD1lo4"
app.config["MAIL_DEFAULT_SENDER"] = "insuresafe67@gmail.com"

mail = Mail(app)

# Serializer for token generation
serializer = URLSafeTimedSerializer(app.secret_key)

# Ensure upload dirs exist
os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)
init_db() # ensures default admin exists

# ---- AI Services ----
face_verifier = FaceVerifier(model_name="ArcFace")
doc_verifier = DocumentVerifier(demo=app.config["DEMO_MODE"])
claims_ai = ClaimsAI(demo=app.config["DEMO_MODE"])

# =====
# AUTH HELPERS
# =====

def login_required(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            return redirect(url_for("login"))
        if not session.get("face_verified") and session.get("role") != "admin":
            return redirect(url_for("face_auth"))
        return f(*args, **kwargs)
    return wrapper

def admin_required(f):
    @wraps(f)
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
def wrapper(*args, **kwargs):
    if not session.get("user_id"):
        return redirect(url_for("login"))
    user = get_user_by_id(session["user_id"])
    if not user or not user.get("is_admin"):
        abort(403)
    return f(*args, **kwargs)
return wrapper

def detect_claim_type(policy, saved_files):
    """
    Detect whether claim is vehicle-based or normal
    """
    # Policy-based detection
    if policy.get("category", "").lower() in ("car", "bike"):
        return "vehicle"

    # File-based detection
    image_exts = (".jpg", ".jpeg", ".png")
    image_files = [f for f in saved_files if f.lower().endswith(image_exts)]

    if image_files and len(image_files) == len(saved_files):
        return "vehicle"

    return "normal"

# =====
# UPLOADS SERVING ROUTE
# =====
from flask import send_from_directory

@app.route('/uploads/<path:filename>')
def uploaded_file(filename):
    return send_from_directory(app.config['UPLOAD_FOLDER'], filename,
as_attachment=False)

@app.errorhandler(RequestEntityTooLarge)
def handle_large_file(e):
    return "Uploaded image is too large", 413

@app.route("/admin/files/<path:filename>")
@admin_required
@login_required
def admin_files(filename):
    # 🔥 FIX: normalize Windows paths
    filename = filename.replace("\\", "/")
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
    return send_from_directory(
        app.config["UPLOAD_FOLDER"],
        filename,
        as_attachment=False
    )

@app.route("/admin/policy-docs/<user_policy_id>")
@login_required
@admin_required
def admin_view_policy_docs(user_policy_id):
    up = user_policies_col.find_one({"_id": ObjectId(user_policy_id)})

    if not up:
        abort(404)

    return render_template(
        "admin_policy_docs.html",
        docs=up.get("uploaded_docs", {})
    )

# =====
# ROUTES: INDEX
# =====
@app.route("/")
def index():
    return render_template("index.html")

@app.route("/get-started")
def get_started():
    return redirect(url_for("login"))

# =====
# ROUTES: AUTH
# =====
@app.route("/auth/signup", methods=["GET", "POST"])
def signup():
    if request.method == "POST":
        step = request.form.get("step", "send_otp")

        name = request.form.get("name","").strip()
        email = request.form.get("email","").strip().lower()
        password = request.form.get("password","")
        phone = request.form.get("phone","").strip()
        address = request.form.get("address","").strip()
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
age = request.form.get("age")
annual_income = request.form.get("annual_income")
id_photo = request.files.get("id_photo")

# ===== STEP 1: SEND OTP =====

if step == "send_otp":

    if not (name and email and password and id_photo):
        flash("All fields including ID photo are required.", "warning")
        return redirect(url_for("signup"))

    if get_user_by_email(email):
        flash("Email already registered.", "danger")
        return redirect(url_for("signup"))

    # Save ID photo temporarily
    id_dir = os.path.join(app.config["UPLOAD_FOLDER"], "temp_ids")
    os.makedirs(id_dir, exist_ok=True)

    filename = secure_filename(f"{int(time.time())}_{id_photo.filename}")
    temp_path = os.path.join(id_dir, filename)
    id_photo.save(temp_path)

    # Store signup data temporarily in session
    session["signup_data"] = {
        "name": name,
        "email": email,
        "password": password,
        "phone": phone,
        "address": address,
        "age": age,
        "annual_income": annual_income,
        "id_photo_path": temp_path
    }

    # 📧 SEND OTP (Brevo)
    otp_service.send_otp(mail, email, purpose="signup")

    flash("OTP sent to your email.", "success")
    return render_template(
        "auth_signup.html",
        otp_sent=True,
        email=email
    )

# ===== STEP 2: VERIFY OTP =====
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
elif step == "verify_otp":
    otp = request.form.get("otp", "").strip()
    data = session.get("signup_data")

    if not data:
        flash("Session expired. Please sign up again.", "danger")
        return redirect(url_for("signup"))

    ok, msg = otp_service.verify_otp(
        data["email"],
        otp,
        purpose="signup"
    )

    if not ok:
        flash(msg, "danger")
        return render_template(
            "auth_signup.html",
            otp_sent=True,
            email=data["email"]
        )

    # Move ID photo to final folder
    final_dir = os.path.join(app.config["UPLOAD_FOLDER"], "id_photos")
    os.makedirs(final_dir, exist_ok=True)

    final_name = os.path.basename(data["id_photo_path"])
    final_path = os.path.join(final_dir, final_name)
    os.rename(data["id_photo_path"], final_path)

    try:
        add_user(
            name=data["name"],
            email=data["email"],
            password=data["password"],
            id_photo_path=f"id_photos/{final_name}",
            phone=data["phone"],
            address=data["address"],
            age=int(data["age"]) if data["age"] else None,
            annual_income=float(data["annual_income"]) if data["annual_income"] else None,
            is_admin=False,
            email_verified=True # ✅ VERIFIED VIA OTP
        )

    session.pop("signup_data", None)
    flash("Signup successful. Please login.", "success")
```


Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        return redirect(url_for("login"))

    except Exception as e:
        flash(f"Signup failed: {e}", "danger")
        return redirect(url_for("signup"))

    return render_template("auth_signup.html")

@app.route("/auth/resend-otp", methods=["POST"])
def resend_otp():
    data = session.get("signup_data")

    if not data:
        return jsonify({"ok": False, "msg": "Session expired"})

    email = data["email"]

    # 🚫 Cooldown check (60s)
    last_sent = session.get("otp_last_sent")
    if last_sent and time.time() - last_sent < 60:
        return jsonify({
            "ok": False,
            "msg": "Please wait before resending OTP"
        })

    send_otp(email, purpose="signup")
    session["otp_last_sent"] = time.time()

    return jsonify({"ok": True, "msg": "OTP resent"})

@app.route("/auth/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")

        user = authenticate_user(email, password)

        if not user:
            flash("Invalid credentials.", "danger")
            return redirect(url_for("login"))

        # ✅ Ensure account is active
        if not user.get("is_active", True):
            flash("Your account is inactive. Contact admin.", "danger")
            return redirect(url_for("login"))
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# ✅ Email verification check (admins bypass)
if not user.get("is_admin") and not user.get("email_verified", False):
    flash("Please verify your email before logging in.", "warning")
    return redirect(url_for("login"))

# 🛡️ Clear old session (safety)
session.clear()

# ----- LOGIN CONTINUES -----
session["role"] = "admin" if user.get("is_admin") else "user"
session["user_id"] = str(user["_id"])

if user.get("is_admin"):
    session["face_verified"] = True
    flash("Admin login successful.", "success")
    return redirect(url_for("admin"))

# 👤 Normal user → face auth
session["face_verified"] = False
session["pending_user_id"] = str(user["_id"])
flash("Password OK. Please complete face verification.", "info")
return redirect(url_for("face_auth"))

return render_template("auth_login.html")

# =====
# ROUTES: FORGET PASSWORD
# =====
@app.route("/auth/forgot", methods=["GET", "POST"])
def forgot():
    if request.method == "POST":
        step = request.form.get("step", "send_otp")
        email = request.form.get("email", "").strip().lower()

        # =====
        # STEP 1: SEND OTP
        # =====
        if step == "send_otp":
            if not email:
                flash("Please enter your email.", "warning")
                return redirect(url_for("forgot"))

            user = get_user_by_email(email)

            # 🛡️ Security: don't reveal if user exists
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
if user:
    otp_service.send_otp(mail, email, purpose="forgot")

flash("If this email exists, an OTP has been sent.", "info")

return render_template(
    "auth_forgot.html",
    otp_sent=True,
    email=email
)

# =====
# STEP 2: VERIFY OTP
# =====
elif step == "verify_otp":
    otp = request.form.get("otp", "").strip()

    ok, msg = otp_service.verify_otp(
        email,
        otp,
        purpose="forgot"
    )

    if not ok:
        flash(msg, "danger")
        return render_template(
            "auth_forgot.html",
            otp_sent=True,
            email=email
        )

    # ✅ OTP VERIFIED → ALLOW PASSWORD RESET
    session["reset_email"] = email
    return redirect(url_for("reset_password"))

return render_template("auth_forgot.html", otp_sent=False)

@app.route("/auth/reset-password", methods=["GET", "POST"])
def reset_password():
    email = session.get("reset_email")
    if not email:
        flash("Session expired. Please try again.", "danger")
        return redirect(url_for("forgot"))

    if request.method == "POST":
        password = request.form.get("password")
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
confirm = request.form.get("confirm")

if not password or not confirm:
    flash("Please fill all fields.", "warning")
    return redirect(request.url)

if password != confirm:
    flash("Passwords do not match.", "danger")
    return redirect(request.url)

user = get_user_by_email(email)
if not user:
    flash("User not found.", "danger")
    return redirect(url_for("forgot"))

# 🛡️ Update password securely
pwd_hash, salt = hash_password(password)
users_col.update_one(
    {"_id": user["_id"]},
    {"$set": {
        "password_hash": pwd_hash,
        "salt": salt
    }}
)

session.pop("reset_email", None)
flash("Password reset successful. Please login.", "success")
return redirect(url_for("login"))

return render_template("auth_forgot.html", reset=True, email=email)

# =====
# FACE AUTH
# =====
@app.route("/auth/face", methods=["GET", "POST"])
def face_auth():
    uid = session.get("pending_user_id")
    if not uid:
        return redirect(url_for("login"))

    result = None

    if request.method == "POST":
        captured_image = request.form.get("captured_image")

        if not captured_image:
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
flash("Camera image not captured.", "warning")
return redirect(url_for("face_auth"))

try:
    user = get_user_by_id(uid)

    if not user or not user.get("id_photo_path"):
        flash("ID photo not found. Please contact support.", "danger")
        return redirect(url_for("login"))

    # ----- BASE64 → IMAGE -----
    img_data = captured_image.split(",")[1]
    decoded_img = base64.b64decode(img_data)

    img = Image.open(BytesIO(decoded_img))
    img = img.convert("RGB")
    img = img.resize((640, 640)) # ✅ IMPORTANT

    # ----- SAVE SELFIE -----
    sf_dir = os.path.join(app.config["UPLOAD_FOLDER"], "selfies")
    os.makedirs(sf_dir, exist_ok=True)

    filename = secure_filename(f"{int(time.time())}_selfie.jpg")
    sf_path = os.path.join(sf_dir, filename)

    img.save(sf_path, "JPEG", quality=85)

    # ----- FIX STORED PATH -----
    stored_path = os.path.join(
        app.config["UPLOAD_FOLDER"],
        user["id_photo_path"].replace("\\", "/")
    )

    print("ID photo:", stored_path)
    print("Selfie:", sf_path)

    if not os.path.exists(stored_path):
        flash("Stored ID image missing.", "danger")
        return redirect(url_for("login"))

    # ----- FACE COMPARE (SAFE) -----
    try:
        result = face_verifier.compare(stored_path, sf_path)
    except Exception as fe:
        print("🔥 Face model error:", fe)
        flash("Face detection failed. Ensure your face is clearly visible.", "danger")
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        return redirect(url_for("face_auth"))

    if not result or result.get("error"):
        flash("No face detected. Try again in good lighting.", "warning")
        return redirect(url_for("face_auth"))

    if result.get("match"):
        session["user_id"] = uid
        session["face_verified"] = True
        session.pop("pending_user_id", None)

        flash("✅ Face verification successful!", "success")
        return redirect(url_for("dashboard"))

    flash("❌ Face mismatch. Try again.", "danger")

except Exception as e:
    print("🔥 Face Auth Fatal Error:", e)
    flash("Unexpected error during face verification.", "danger")

return render_template("auth_face.html", result=result)

# =====
# LOGOUT
# =====
@app.route("/logout")
def logout():
    session.clear()
    flash("Logged out.", "info")
    return redirect(url_for("login"))

# =====
# POLICIES (User-facing)
# =====
@app.route("/policies")
@login_required
def policies():
    uid = session["user_id"]
    user = get_user_by_id(uid)

    if not user:
        flash("User not found.", "danger")
        return redirect(url_for("logout"))

# -----
# User features with defaults
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# -----
age = int(user.get("age", 30))
annual_income = int(user.get("annual_income", 0)) # IMPORTANT

# -----
# User's existing policies
# -----
existing_policies = get_user_policies(uid) or []
existing_policy_ids = [
    str(p.get("policy_id")) for p in existing_policies if p.get("policy_id")
]

# -----
# Compute total premium
# -----
total_premium = 0.0
if existing_policy_ids:
    try:
        object_ids = [ObjectId(pid) for pid in existing_policy_ids]
        cursor = policies_col.find({"_id": {"$in": object_ids}})
        total_premium = sum(float(p.get("premium_amount", 0)) for p in cursor)
    except Exception:
        total_premium = 0.0

# -----
# Get recommended policies
# -----
recommended = recommend_policies(
    age=age,
    annual_income=annual_income,
    existing_premiums=total_premium,
    existing_policy_count=len(existing_policy_ids)
) or []

# -----
# Fetch all policies
# -----
all_policies = list(policies_col.find())

# 🔥 FIX: Normalize income fields to avoid NoneType errors
for p in all_policies:
    p["min_income"] = int(p.get("min_income") or 0)
    p["max_income"] = int(p.get("max_income") or 10**9)
    p["premium_amount"] = float(p.get("premium_amount") or 0)

# -----
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# Map policy names to DB docs
# -----
name_to_doc = {
    (p.get("name") or "").strip().lower(): p
    for p in all_policies
}

# -----
# Map recommender output to DB _id and score
# -----
recommended_ids = []
scores = {}

for r in recommended:
    rec_name = (r.get("name") or "").strip().lower()
    matched = name_to_doc.get(rec_name)
    if matched:
        pid = str(matched["_id"])
        recommended_ids.append(pid)
        scores[pid] = float(r.get("score", 0))

# -----
# Sort policies
# -----
def sort_key(p):
    pid = str(p["_id"])
    if pid in existing_policy_ids and pid in recommended_ids:
        return (0, -scores.get(pid, 0))
    elif pid in recommended_ids:
        return (1, -scores.get(pid, 0))
    else:
        return (2, 0)

all_policies.sort(key=sort_key)

# -----
# Render
# -----
return render_template(
    "policies.html",
    policies=all_policies,
    recommended_ids=recommended_ids,
    scores=scores,
    existing_policy_ids=existing_policy_ids,
    user=user # ✅ REQUIRED for income==0 UI logic
)
```


Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# =====
# APPLY POLICY
# =====
@app.route("/apply/<pid>", methods=["GET", "POST"])
@login_required
def apply_policy(pid):

    # ♦ Fetch logged-in user
    user = get_user_by_id(session["user_id"])

    # ♦ Block application if income is 0
    if user.get("annual_income", 0) == 0:
        flash("⚠ You cannot apply for policies with zero income.", "warning")
        return redirect(url_for("policies"))

    policy = get_policy_by_id(pid)
    if not policy:
        abort(404)

    result = None

    CATEGORY_REQUIREMENTS = {
        "Health": ["aadhar_card", "pan_card", "medical_records"],
        "Car": ["aadhar_card", "pan_card", "driving_license", "vehicle_registration", "legal_rc"],
        "Life": ["aadhar_card", "pan_card", "income_proof", "medical_records"],
        "Bike": ["aadhar_card", "pan_card", "driving_license", "bike_registration", "legal_rc"],
        "Family": ["aadhar_card", "pan_card", "family_details"]
    }

    REQUIREMENT_LABELS = {
        "aadhar_card": "Aadhar Card",
        "pan_card": "PAN Card",
        "medical_records": "Medical Certificate / Records",
        "income_proof": "Income Proof",
        "driving_license": "Driving License",
        "vehicle_registration": "Vehicle Registration",
        "bike_registration": "Bike Registration",
        "legal_rc": "Legal RC",
        "family_details": "Family Member Details"
    }

    required_fields = CATEGORY_REQUIREMENTS.get(
        policy.get("category", "General"),
        ["aadhar_card", "pan_card"]
    )
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
requirements = {
    field: REQUIREMENT_LABELS[field]
    for field in required_fields
}

if request.method == "POST":

    form_data = {
        "name": request.form.get("name", "").strip(),
        "dob": request.form.get("dob", "").strip(),
        "gender": request.form.get("gender", "").strip(),
        "email": request.form.get("email", "").strip(),
        "phone": request.form.get("phone", "").strip(),
        "address": request.form.get("address", "").strip()
    }

    uploaded_docs = {}

    # ✅ RELATIVE PATH (stored in DB)
    relative_dir = os.path.join(
        "documents",
        str(session["user_id"])
    )

    # ✅ ABSOLUTE PATH (actual disk location)
    absolute_dir = os.path.join(
        app.config["UPLOAD_FOLDER"],
        relative_dir
    )

    os.makedirs(absolute_dir, exist_ok=True)

    for field in required_fields:
        file = request.files.get(field)

        if not file or not file.filename:
            flash(f"Please upload: {REQUIREMENT_LABELS[field]}", "warning")
            return redirect(request.url)

        filename = secure_filename(f"{int(time.time())}_{file.filename}")

        # 💡 Paths
        relative_path = os.path.join(relative_dir, filename)
        absolute_path = os.path.join(app.config["UPLOAD_FOLDER"], relative_path)
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
file.save(absolute_path)

# ✅ STORE RELATIVE PATH ONLY
uploaded_docs[field] = os.path.join(
"documents",
str(session["user_id"]),
filename
)

validation_data = {
    "name": form_data["name"],
    "dob": form_data["dob"],
    "gender": form_data["gender"]
}

primary_doc_key = "aadhar_card"
result = doc_verifier.validate(
    os.path.join(app.config["UPLOAD_FOLDER"], uploaded_docs[primary_doc_key]),
    validation_data
)

valid = bool(result.get("is_valid"))

assign_policy_to_user(
    user_id=session["user_id"],
    policy_id=pid,
    status="active" if valid else "pending",
    doc_valid=valid,
    uploaded_docs=uploaded_docs
)

if valid:
    flash("✅ Application verified. Policy activated.", "success")
else:
    flash("⚠ Document mismatch. Sent for manual review.", "warning")

return redirect(url_for("dashboard"))

return render_template(
    "apply_policy.html",
    policy=policy,
    requirements=requirements,
    result=result,
    user=user
)
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# =====
# DASHBOARD & CLAIMS
# =====

from datetime import datetime, timedelta
from bson import ObjectId

@app.route("/dashboard")
@login_required
def dashboard():
    uid = session["user_id"]
    now = datetime.utcnow()

    # -----
    # User policies
    # -----
    policies = get_user_policies(uid)

    # -----
    # Claims with policy join
    # -----
    claims = list(claims_col.aggregate([
        {"$match": {"user_id": ObjectId(uid)}},
        {"$lookup": {
            "from": "policies",
            "localField": "policy_id",
            "foreignField": "_id",
            "as": "policy"
        }},
        {"$unwind": "$policy"},
        {"$sort": {"created_at": -1}}
    ]))

    # -----
    # Latest claim + cooldown map
    # -----
    claim_map = {}
    cooldown_map = {}

    # -----
    # Mini claim summary per policy
    # -----
    claim_summary = {}

    for c in claims:
        pid = str(c["policy_id"])
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# Init summary
if pid not in claim_summary:
    claim_summary[pid] = {
        "approved": 0,
        "pending": 0,
        "rejected": 0
    }

status = c.get("status")
if status in claim_summary[pid]:
    claim_summary[pid][status] += 1

# Latest claim per policy
if pid not in claim_map:
    claim_map[pid] = c

# Cooldown only if rejected
if status == "rejected":
    cooldown_map[pid] = c["created_at"] + timedelta(days=30)

return render_template(
    "dashboard.html",
    policies=policies,
    claims=claims,
    claim_map=claim_map,
    claim_summary=claim_summary, # ✅ NEW
    cooldown_map=cooldown_map, # ✅ FIXED
    now=now # ✅ FIXED
)

# =====
# APPLY CLAIM (Dedicated AI Claims Page)
# =====
@app.route("/claim/apply/<policy_id>", methods=["GET", "POST"])
@login_required
def apply_claim(policy_id):

    policy = get_policy_by_id(policy_id)
    if not policy:
        abort(404)

    claim_type = request.args.get("type", "vehicle")
    stage = None
    report = None
    files = [] # ✅ SAFE DEFAULT
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
if request.method == "POST":
    claim_type = request.form.get("claim_type")
    stage = request.form.get("stage")

    # =====
    # 🚗 VEHICLE CLAIM
    # =====
    if claim_type == "vehicle":

        # ----- STAGE 1: DAMAGE ESTIMATION -----
        if stage == "estimate":
            files = request.files.getlist("claim_files")

            if not files or not files[0].filename:
                flash("Please upload at least one vehicle damage image.", "warning")
                return redirect(request.url)

            # ❌ Block PDFs
            for f in files:
                if f.filename.lower().endswith(".pdf"):
                    flash("Vehicle claims cannot include PDFs.", "danger")
                    return redirect(request.url)

            image_paths = []
            c_dir = os.path.join(app.config["UPLOAD_FOLDER"], "claims")
            os.makedirs(c_dir, exist_ok=True)

            for f in files:
                filename = secure_filename(f"{int(time.time())}_{f.filename}")
                abs_path = os.path.join(c_dir, filename)
                f.save(abs_path)
                image_paths.append(abs_path)

            report = claims_ai.evaluate_vehicle_damage(
                image_paths=image_paths,
                vehicle_type=policy.get("category", "").lower()
            )

        # ----- STAGE 2: BARGAIN -----
        elif stage == "bargain":
            ai_estimate = float(request.form.get("ai_estimate", 0))
            claim_amount = float(request.form.get("claim_amount", 0))

            report = claims_ai.evaluate_bargain(ai_estimate, claim_amount)
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
add_claim(
    user_id=session["user_id"],
    policy_id=policy_id,
    amount=claim_amount,
    status="pending",
    risk_score=report.get("risk_score"),
    decision=report.get("decision"),
    claim_type="vehicle"
)

flash("🚗 Vehicle claim submitted successfully.", "success")
return redirect(url_for("dashboard"))

else:
    flash("Invalid vehicle claim stage.", "danger")
    return redirect(request.url)

# =====
# 📄 DOCUMENT CLAIM
# =====
elif claim_type == "document":
    files = request.files.getlist("claim_files")
    claim_amount = float(request.form.get("claim_amount", 0))

    if not files or not files[0].filename:
        flash("Please upload claim documents.", "warning")
        return redirect(request.url)

    abs_paths = [] # for AI
    rel_paths = [] # for DB + admin

    c_dir = os.path.join(app.config["UPLOAD_FOLDER"], "claims")
    os.makedirs(c_dir, exist_ok=True)

    for f in files:
        filename = secure_filename(f"{int(time.time())}_{f.filename}")
        rel_path = f"claims/{filename}"
        abs_path = os.path.join(app.config["UPLOAD_FOLDER"], rel_path)

        f.save(abs_path)
        abs_paths.append(abs_path)
        rel_paths.append(rel_path)

    report = claims_ai.evaluate(
        files=abs_paths,
        metadata={
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        "claim_amount": claim_amount,
        "policy_sum_insured": policy.get("sum_insured", 100000)
    }
)

add_claim(
    user_id=session["user_id"],
    policy_id=policy_id,
    amount=claim_amount,
    status="pending",
    risk_score=report.get("risk_score"),
    decision=report.get("decision"),
    claim_type="document",
    uploaded_docs=rel_paths # ✅ ADMIN CAN VIEW
)

flash("📄 Document claim submitted. Review in progress.", "success")
return redirect(url_for("dashboard"))

else:
    flash("Invalid claim type.", "danger")
    return redirect(request.url)

return render_template(
    "apply_claim.html",
    policy=policy,
    report=report,
    claim_type=claim_type
)

# =====
# PROFILE
# =====
@app.route("/profile")
@login_required
def profile():
    uid = session["user_id"]
    user = get_user_by_id(uid)

    # ✅ Fix backslashes in stored image paths
    if user and user.get("id_photo_path"):
        user["id_photo_path"] = user["id_photo_path"].replace("\\", "/")

    policies = get_user_policies(uid)
    return render_template("profile.html", user=user, policies=policies)
```


Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
@app.route("/profile/edit", methods=["POST"])
@login_required
def edit_profile():
    uid = session["user_id"]

    name = request.form.get("name")
    email = request.form.get("email")
    phone = request.form.get("phone")
    address = request.form.get("address")
    annual_income = request.form.get("annual_income")

    update_data = {
        "name": name,
        "email": email,
        "phone": phone,
        "address": address,
        "annual_income": int(annual_income or 0) #  SAFE
    }

    # Handle ID photo upload (unchanged logic)
    file = request.files.get("id_photo")
    if file and file.filename:
        filename = secure_filename(file.filename)
        path = os.path.join("id_photos", f"{uid}_{filename}")
        file.save(os.path.join(app.config["UPLOAD_FOLDER"], path))
        update_data["id_photo_path"] = path

    users_col.update_one(
        {"_id": ObjectId(uid)},
        {"$set": update_data}
    )

    user = get_user_by_id(uid)

    return jsonify({
        "status": "success",
        "message": "Profile updated successfully.",
        "user": {
            "name": user["name"],
            "email": user["email"],
            "phone": user.get("phone"),
            "address": user.get("address"),
            "annual_income": user.get("annual_income", 0),
            "id_photo_path": user.get("id_photo_path")
        }
    })
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# =====
# ADMIN
# =====
@app.route("/admin")
@admin_required
def admin():
    try:
        users = list(users_col.find())

        for u in users:
            if "email_verified" not in u:
                u["email_verified"] = False

        policy_defs = list(policies_col.find())

        enriched_claims = list(claims_col.aggregate([
            {
                "$lookup": {
                    "from": "users",
                    "localField": "user_id",
                    "foreignField": "_id",
                    "as": "user"
                }
            },
            { "$unwind": "$user" },
            {
                "$lookup": {
                    "from": "policies",
                    "localField": "policy_id",
                    "foreignField": "_id",
                    "as": "policy"
                }
            },
            { "$unwind": "$policy" },
            { "$sort": { "created_at": -1 } }
        ]))

        for c in enriched_claims:
            c.setdefault("claim_type", "normal")

# ✅ USER POLICIES (DOCUMENT REVIEW)
user_policies = list(user_policies_col.aggregate([
    {
        "$lookup": {
            "from": "users",
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        "localField": "user_id",
        "foreignField": "_id",
        "as": "user"
    }
},
{ "$unwind": "$user" },
{
    "$lookup": {
        "from": "policies",
        "localField": "policy_id",
        "foreignField": "_id",
        "as": "policy"
    }
},
{ "$unwind": "$policy" },
{ "$sort": { "applied_at": -1 } }
]))

counts = {
    "n_users": users_col.count_documents({}),
    "n_up": policies_col.count_documents({}),
    "n_c": claims_col.count_documents({}),
}

return render_template(
    "admin.html",
    users=users,
    policy_defs=policy_defs,
    claims=enriched_claims,
    user_policies=user_policies,
    counts=counts
)

except Exception as e:
    flash(f"Admin load failed: {e}", "danger")
    return render_template(
        "admin.html",
        users=[],
        policy_defs=[],
        claims=[],
        user_policies=[],
        counts={"n_users": 0, "n_up": 0, "n_c": 0}
    )

@app.route("/admin/verify_email/<uid>")
@admin_required
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
def admin_verify_email(uid):
    try:
        users_col.update_one(
            {"_id": ObjectId(uid)},
            {"$set": {"email_verified": True}}
        )
        flash("User email verified successfully.", "success")
    except Exception as e:
        flash(f"Verification failed: {e}", "danger")

    return redirect(url_for("admin"))

@app.route("/admin/update_policy_status/<policy_id>/<status>")
@admin_required
def update_policy_status(policy_id, status):
    try:
        update_user_policy_status(
            policy_id,
            status,
            doc_valid=status.lower() in ("approved", "active")
        )
        flash("Policy status updated.", "success")
    except Exception as e:
        flash(f"Failed to update policy: {e}", "danger")

    return redirect(url_for("admin"))

@app.route("/admin/update_claim_status/<claim_id>/<status>")
@admin_required
def update_claim_status(claim_id, status):
    try:
        status = status.lower()

        if status == "approved":
            decision = "Auto-Approve"
        elif status == "manual":
            decision = "Manual Review"
        else:
            decision = "Rejected"

        db_update_claim_status(claim_id, status, decision)
        flash("Claim status updated.", "success")

    except Exception as e:
        flash(f"Failed to update claim: {e}", "danger")
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        return redirect(url_for("admin"))

@app.route("/admin/verify_policy/<user_policy_id>")
@admin_required
def admin_verify_policy(user_policy_id):
    try:
        update_user_policy_status(
            user_policy_id=user_policy_id,
            status="active",
            doc_valid=True
        )
        flash("Policy documents verified and policy activated.", "success")
    except Exception as e:
        flash(f"Policy verification failed: {e}", "danger")

    return redirect(url_for("admin"))

@app.route("/admin/toggle/<uid>")
@admin_required
def admin_toggle(uid):
    try:
        new_status = toggle_user_active(uid)
        if new_status is not None:
            flash(
                f"User {'activated' if new_status else 'deactivated'}.",
                "success"
            )
        else:
            flash("User not found.", "warning")
    except Exception as e:
        flash(f"Toggle failed: {e}", "danger")

    return redirect(url_for("admin"))

# =====
# POLICY MANAGEMENT (Admin)
# =====
@app.route("/admin/policy/add", methods=["POST"])
@admin_required
def add_policy():
    try:
        data = request.form
        policy = {
            "name": data.get("name"),
            "category": data.get("category"),
            "description": data.get("description"),
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
"requirements": data.get("requirements", ""),
"min_age": int(data.get("min_age", 0)),
"max_age": int(data.get("max_age", 100)),
"min_income": float(data.get("min_income", 0)),
"max_income": float(data.get("max_income", 0)),
"premium_amount": float(data.get("premium_amount", 0)),
"duration_years": int(data.get("duration_years", 1)),
"created_at": datetime.utcnow()
}
policies_col.insert_one(policy)
flash("Policy added successfully.", "success")
except Exception as e:
    flash(f"Failed to add policy: {e}", "danger")

return redirect(url_for("admin"))

@app.route("/admin/policy/delete/<policy_id>", methods=["GET","POST"])
@admin_required
def admin_delete_policy(policy_id):
    try:
        oid = ObjectId(policy_id)
    except:
        oid = policy_id
    try:
        res = policies_col.delete_one({"_id": oid})
        if res.deleted_count:
            flash("Policy deleted.", "success")
        else:
            flash("Policy not found.", "warning")
    except Exception as e:
        flash(f"Failed to delete policy: {e}", "danger")
    return redirect(url_for("admin"))

# =====
# TEMPLATE FILTERS
# =====
@app.template_filter("datetimeformat")
def datetimeformat(value, format="%Y-%m-%d %H:%M"):
    if isinstance(value, int):
        value = datetime.fromtimestamp(value)
    elif isinstance(value, str):
        try:
            value = datetime.fromisoformat(value)
        except ValueError:
            return value
    elif not isinstance(value, datetime):
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        return value
    return value.strftime(format)

# =====
# MAIN
# =====
if __name__ == "__main__":
    app.run(debug=True, threaded=False)
```

2. db.py

```
# db.py - MongoDB helpers
import os, hashlib, hmac, secrets
from pymongo import MongoClient
from bson.objectid import ObjectId
from datetime import datetime, timedelta # ✅ timedelta needed

# ----- MongoDB Connection ----- #
MONGO_URI = os.environ.get("MONGO_URI", "mongodb://localhost:27017/")
client = MongoClient(MONGO_URI)
db = client["insure_db"]

# ----- Collections ----- #
users_col = db["users"]
policies_col = db["policies"]
user_policies_col = db["user_policies"]
claims_col = db["claims"]
otp_col = db["otps"] # ✅ OTP collection

# ----- Password Hashing ----- #
def hash_password(password, salt=None):
    if salt is None:
        salt = secrets.token_hex(16)
    pwd_hash = hmac.new(
        salt.encode(),
        password.encode(),
        hashlib.sha256
    ).hexdigest()
    return pwd_hash, salt

def verify_password(stored_hash, stored_salt, password_attempt):
    attempt_hash, _ = hash_password(password_attempt, stored_salt)
    return hmac.compare_digest(stored_hash, attempt_hash)

# ----- DB INIT ----- #
def init_db():
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask, Frontend:

HTML, CSS, JavaScript

Database: MongoDB

```
# USERS
users_col.create_index("email", unique=True)

# USER POLICIES
user_policies_col.create_index([("user_id", 1), ("policy_id", 1)])

# CLAIMS
claims_col.create_index("created_at")
claims_col.create_index([("user_id", 1), ("policy_id", 1)])

# OTP ( 🔥 IMPORTANT)
otp_col.create_index("email")
otp_col.create_index(
    "expires_at",
    expireAfterSeconds=0 # auto-delete expired OTPs
)

# DEFAULT ADMIN
create_default_admin()

# ----- User Functions ----- #
def add_user(
    name,
    email,
    password,
    id_photo_path=None,
    phone=None,
    address=None,
    age=None,
    annual_income=None,
    is_admin=False,
    email_verified=False # ✅ VERIFIED ONLY AFTER OTP
):
    pwd_hash, salt = hash_password(password)
    user = {
        "name": name,
        "email": email,
        "password_hash": pwd_hash,
        "salt": salt,
        "id_photo_path": id_photo_path,
        "phone": phone,
        "address": address,
        "age": int(age) if age else None,
        "annual_income": int(annual_income) if annual_income is not None else 0,
        "is_admin": is_admin,
        "is_active": True,
```


Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
"email_verified": email_verified, # 🚫 IMPORTANT
"created_at": datetime.utcnow()
}
return str(users_col.insert_one(user).inserted_id)

def get_user_by_email(email):
    return users_col.find_one({"email": email})

def get_user_by_id(user_id):
    if isinstance(user_id, str):
        user_id = ObjectId(user_id)
    return users_col.find_one({"_id": user_id})

def authenticate_user(email, password):
    user = get_user_by_email(email)
    if user and verify_password(user["password_hash"], user["salt"], password):
        return user
    return None

def toggle_user_active(uid):
    try:
        oid = ObjectId(uid)
    except:
        oid = uid

    user = users_col.find_one({"_id": oid})
    if not user:
        return None

    new_status = not user.get("is_active", True)
    users_col.update_one(
        {"_id": oid},
        {"$set": {"is_active": new_status}}
    )
    return new_status

# ----- Policy Functions ----- #
def add_policy(name, description, requirements="", sum_insured=0):
    policy = {
        "name": name,
        "description": description,
        "requirements": requirements,
        "sum_insured": sum_insured,
        "created_at": datetime.utcnow()
    }
    return policies_col.insert_one(policy).inserted_id
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
def get_policy_by_id(pid):
    if isinstance(pid, str):
        pid = ObjectId(pid)
    return policies_col.find_one({"_id": pid})

# ----- User-Policy Functions ----- #
def assign_policy_to_user(
    user_id,
    policy_id,
    status="pending",
    doc_valid=False,
    uploaded_docs=None
):
    if isinstance(user_id, str):
        user_id = ObjectId(user_id)
    if isinstance(policy_id, str):
        policy_id = ObjectId(policy_id)

    record = {
        "user_id": user_id,
        "policy_id": policy_id,
        "status": status,
        "doc_valid": doc_valid,
        "uploaded_docs": uploaded_docs or {},
        "applied_at": datetime.utcnow()
    }
    return user_policies_col.insert_one(record).inserted_id

def get_user_policies(user_id):
    if isinstance(user_id, str):
        user_id = ObjectId(user_id)

    return list(user_policies_col.aggregate([
        {"$match": {"user_id": user_id}},
        {"$lookup": {
            "from": "policies",
            "localField": "policy_id",
            "foreignField": "_id",
            "as": "policy"
        }},
        {"$unwind": "$policy"},
        {"$sort": {"applied_at": -1}}
    ]))

def update_user_policy_status(user_policy_id, status, doc_valid=None):
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
if isinstance(user_policy_id, str):
    user_policy_id = ObjectId(user_policy_id)

upd = {"status": status}
if doc_valid is not None:
    upd["doc_valid"] = doc_valid

user_policies_col.update_one(
    {"_id": user_policy_id},
    {"$set": upd}
)

# ----- Claims Functions ----- #
def add_claim(
    user_id,
    policy_id,
    amount,
    status="pending",
    risk_score=None,
    decision=None,
    claim_type=None,
    uploaded_docs=None # ✅ ADD THIS
):
    if isinstance(user_id, str):
        user_id = ObjectId(user_id)
    if isinstance(policy_id, str):
        policy_id = ObjectId(policy_id)

    record = {
        "user_id": user_id,
        "policy_id": policy_id,
        "amount": amount,
        "status": status,
        "risk_score": risk_score,
        "decision": decision,
        "claim_type": claim_type or "normal",
        "uploaded_docs": uploaded_docs or [], # ✅ STORE FILES
        "created_at": datetime.utcnow()
    }

    return claims_col.insert_one(record).inserted_id

def db_update_claim_status(claim_id, status, decision=None):
    if isinstance(claim_id, str):
        claim_id = ObjectId(claim_id)
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
upd = {"status": status}
if decision is not None:
    upd["decision"] = decision

claims_col.update_one({"_id": claim_id}, {"$set": upd})

# -----
# 🚫 CLAIM COOLDOWN CHECK (NEW – IMPORTANT)
# -----
def has_recent_claim(user_id, policy_id, days=30):
    """
    Returns True if a claim exists for this policy
    within the last `days`
    """
    if isinstance(user_id, str):
        user_id = ObjectId(user_id)
    if isinstance(policy_id, str):
        policy_id = ObjectId(policy_id)

    since = datetime.utcnow() - timedelta(days=days)

    return claims_col.find_one({
        "user_id": user_id,
        "policy_id": policy_id,
        "created_at": {"$gte": since}
    }) is not None

# ----- Admin Helpers ----- #
def get_all_policies():
    return list(policies_col.find().sort("created_at", -1))

def get_all_users():
    return list(users_col.find({}, {"password_hash": 0, "salt": 0}))

# ----- Default Admin Setup ----- #
def create_default_admin():
    admin_email = "admin@mail.com"
    admin_pass = "admin123"

    if not users_col.find_one({"email": admin_email}):
        pwd_hash, salt = hash_password(admin_pass)
        users_col.insert_one({
            "name": "System Admin",
            "email": admin_email,
            "password_hash": pwd_hash,
            "salt": salt,
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
"is_admin": True,
"is_active": True,
"email_verified": True, #  ADMIN ALWAYS VERIFIED
"created_at": datetime.utcnow()
})
print(f"[INFO] Default admin created → {admin_email} / {admin_pass}")
```

3. claims_ai.py

```
# services/claims_ai.py
# AI-assisted claims processing
# NORMAL + VEHICLE (MULTI-IMAGE) + BARGAIN

import hashlib
from typing import Dict, Any, List, Union

class ClaimsAI:
    def __init__(self, model_path: str = None, demo: bool = True):
        self.demo = demo
        self.model_path = model_path

    # -----
    # Utility
    # -----
    def _stable_random(self, *args) -> float:
        """
        Stable pseudo-random value for demo heuristics
        """
        m = hashlib.sha256(":".join(map(str, args)).encode()).hexdigest()
        return int(m[:8], 16) / 0xFFFFFFFF

    # -----
    # DOCUMENT CLAIM (NORMAL)
    # -----
    def evaluate(
        self,
        files: List[str],
        metadata: Dict[str, Any]
    ) -> Dict[str, Any]:

        base = sum(self._stable_random(f) for f in files) / max(1, len(files))

        claimed_amt = float(metadata.get("claim_amount", 0))
        sum_insured = float(metadata.get("policy_sum_insured", 100000))
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
amt_ratio = min(1.0, claimed_amt / (sum_insured + 1e-6))
```

```
risk = 0.35 * base + 0.65 * amt_ratio
```

```
decision = (  
    "Auto-Approve" if risk < 0.35  
    else "Manual Review" if risk < 0.65  
    else "Reject"  
)
```

```
return {  
    "risk_score": round(risk, 3),  
    "decision": decision,  
    "signals": {  
        "claim_type": "normal",  
        "amount_ratio": round(amt_ratio, 3),  
        "documents_used": len(files),  
        "file_quality_proxy": round(base, 3),  
        "rules": [  
            "Amount vs Sum Insured",  
            "Document consistency",  
            "Completeness check"  
        ]  
    }  
}
```

```
# -----  
# VEHICLE DAMAGE ESTIMATION (MULTI IMAGE)  
# -----
```

```
def evaluate_vehicle_damage(  
    self,  
    image_paths: Union[str, List[str]],  
    vehicle_type: str  
) -> Dict[str, Any]:  
    """
```

```
    STEP 1:  
    - Runs YOLO per image  
    - Aggregates results  
    - NO decision here  
    """
```

```
from services.vehicle_damage_ai import assess_vehicle_damage
```

```
# Normalize input  
if isinstance(image_paths, str):  
    image_paths = [image_paths]
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
total_cost = 0
all_damages = []
breakdown = []

for img in image_paths:
    result = assess_vehicle_damage(img, vehicle_type)

    total_cost += result.get("estimated_cost", 0)
    all_damages.extend(result.get("detected_damages", []))
    breakdown.extend(result.get("breakdown", []))

# Average across images
avg_cost = int(total_cost / max(1, len(image_paths)))

severity = (
    "low" if avg_cost < 10000
    else "medium" if avg_cost < 30000
    else "high"
)

return {
    "estimated_damage": avg_cost,
    "severity": severity,
    "signals": {
        "claim_type": "vehicle",
        "vehicle_type": vehicle_type,
        "images_used": len(image_paths),
        "aggregation": "average",
        "detected_damages": list(set(all_damages)),
        "breakdown": breakdown,
        "model": "YOLO Damage Detection"
    }
}

# -----
# VEHICLE CLAIM BARGAIN (STEP 2)
# -----
def evaluate_bargain(
    self,
    ai_estimate: float,
    user_amount: float
) -> Dict[str, Any]:

    if ai_estimate <= 0:
        return {
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        "decision": "Manual Review",
        "risk_score": 0.60,
        "signals": {
            "reason": "No detectable damage",
            "ai_estimate": ai_estimate,
            "user_amount": user_amount
        }
    }

    deviation = (user_amount - ai_estimate) / ai_estimate

    if deviation <= 0.10:
        decision = "Auto-Approve"
        risk = 0.15
    elif deviation <= 0.30:
        decision = "Manual Review"
        risk = 0.45
    else:
        decision = "Reject"
        risk = 0.80

    return {
        "decision": decision,
        "risk_score": round(risk, 2),
        "signals": {
            "claim_type": "vehicle",
            "ai_estimate": ai_estimate,
            "user_amount": user_amount,
            "deviation_pct": round(deviation * 100, 1),
            "rule": "Deviation-based negotiation"
        }
    }
}

# -----
# BACKWARD COMPATIBILITY WRAPPER
# -----
def evaluate_vehicle_claim(
    self,
    image_path: Union[str, List[str]],
    vehicle_type: str,
    claimed_amount: float
) -> Dict[str, Any]:

    estimate = self.evaluate_vehicle_damage(
        image_paths=image_path,
        vehicle_type=vehicle_type
```


Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
)

ai_cost = estimate.get("estimated_damage", 0)

bargain = self.evaluate_bargain(
    ai_estimate=ai_cost,
    user_amount=claimed_amount
)

return {
    **bargain,
    "estimated_damage": ai_cost,
    "severity": estimate.get("severity"),
    "signals": {
        **estimate.get("signals", {}),
        **bargain.get("signals", {})
    }
}
```

4. face_verify.py

```
from deepface import DeepFace

class FaceVerifier:
    def __init__(self, model_name="ArcFace"):
        self.model_name = model_name

    def compare(self, id_photo_path: str, selfie_path: str):
        try:
            result = DeepFace.verify(
                img1_path=id_photo_path,
                img2_path=selfie_path,
                model_name=self.model_name,
                enforce_detection=True # Auto-detects face
            )

            return {
                "match": result.get("verified", False),
                "similarity": float(result.get("distance", 0.0))
            }

        except Exception as e:
            print("DeepFace Error:", e)
            return {"match": False, "similarity": 0.0}
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

5. ocr_verify.py

```
# services/ocr_verify.py
# Aadhaar-aware OCR & verification service (SAFE & PRODUCTION-READY)

import re
import cv2
from typing import Dict, Any
from PIL import Image

import easyocr
import pytesseract
import os

# -----
# Optional: Windows Tesseract
# -----
if os.name == "nt":
    pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"

class DocumentVerifier:
    def __init__(self, demo: bool = False):
        self.demo = demo

        if not self.demo:
            self.reader = easyocr.Reader(['en'], gpu=False)
        else:
            self.reader = None

# -----
# Image loading
# -----
def _load_image(self, path: str):
    img = cv2.imread(path)
    if img is None:
        raise ValueError("Invalid image file")
    return img

# -----
# OCR extraction
# -----
def extract_text(self, image_path: str) -> str:

    # ♦ DEMO MODE (NO OCR)
    if self.demo:
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        return (
            "Name: Rahul Sharma\n"
            "DOB: 1998-05-12\n"
            "Gender: Male\n"
            "Aadhaar: 1234 5678 9012"
        )

    img = self._load_image(image_path)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    text = ""

    # ♦ EasyOCR
    if self.reader:
        results = self.reader.readtext(img_rgb)
        if results:
            text = "\n".join([r[1] for r in results])

    # ♦ Fallback to Tesseract
    if not text.strip():
        pil = Image.fromarray(img_rgb)
        text = pytesseract.image_to_string(pil)

    return text.strip()

# -----
# Aadhaar field extraction
# -----
def extract_fields(self, text: str) -> Dict[str, str]:
    fields = {}

    lines = [l.strip() for l in text.split("\n") if len(l.strip()) > 3]

    # ---- Name (skip govt headers) ----
    ignore_words = {"government", "authority", "india", "unique"}
    name_candidates = []

    for line in lines:
        lower = line.lower()
        if any(w in lower for w in ignore_words):
            continue
        alpha_ratio = sum(c.isalpha() for c in line) / len(line)
        if alpha_ratio > 0.6:
            name_candidates.append(line)

    if name_candidates:
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
        fields["name"] = max(name_candidates, key=len)

# ---- DOB ----
dob_match = re.search(
    r'(\d{2}[/]\d{2}[/]\d{4})\d{4}[/]\d{2}[/]\d{2})',
    text
)
if dob_match:
    fields["dob"] = dob_match.group(1)

# ---- Gender ----
if re.search(r'\bfemale\b', text, re.I):
    fields["gender"] = "female"
elif re.search(r'\bmale\b', text, re.I):
    fields["gender"] = "male"

# ---- Aadhaar Number (ALL formats) ----
uid_match = re.search(
    r'(\d{4}[\s-]?\d{4}[\s-]?\d{4})',
    text
)
if uid_match:
    fields["aadhaar"] = uid_match.group(1).replace("-", " ")

return fields

# -----
# Normalization helper
# -----
def _norm(self, s: str) -> str:
    return re.sub(r'[^a-z0-9]', '', s.lower()) if s else ""

# -----
# Main validation API
# -----
def validate(self, image_path: str, expected: Dict[str, str]) -> Dict[str, Any]:

    text = self.extract_text(image_path)
    extracted = self.extract_fields(text)

    match = {}

    if extracted.get("name") and expected.get("name"):
        match["name"] = self._norm(extracted["name"]) == self._norm(expected["name"])

    if extracted.get("dob") and expected.get("dob"):
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
match["dob"] = extracted["dob"] == expected["dob"]

if extracted.get("gender") and expected.get("gender"):
    match["gender"] = extracted["gender"] == expected["gender"].lower()

is_valid = bool(match) and all(match.values())

return {
    "document_type": "AADHAAR",
    "extracted": extracted,
    "match": match,
    "is_valid": is_valid
}
```

6. otp_service.py

```
# services/otp_service.py
import random
from datetime import datetime, timedelta
from flask_mail import Message
from db import otp_col

OTP_EXPIRY_MINUTES = 5
MAX_ATTEMPTS = 5

def send_otp(mail, email, purpose="signup"):
    """
    Sends OTP to email using Brevo SMTP
    """

    otp = str(random.randint(100000, 999999))
    expires_at = datetime.utcnow() + timedelta(minutes=OTP_EXPIRY_MINUTES)

    # Remove old OTPs
    otp_col.delete_many({"email": email, "purpose": purpose})

    otp_col.insert_one({
        "email": email,
        "otp": otp,
        "purpose": purpose,
        "attempts": 0,
        "expires_at": expires_at,
        "created_at": datetime.utcnow()
    })
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
msg = Message(
    subject="Your InsureSafe OTP",
    recipients=[email],
    body=f"""\n
Your One-Time Password (OTP) is: {otp}\n
\n
Purpose: {purpose.upper()}\n
Valid for {OTP_EXPIRY_MINUTES} minutes.\n
\n
If you did not request this, ignore this email.\n
"""\n
)

mail.send(msg)

def verify_otp(email, otp, purpose="signup"):
    record = otp_col.find_one({
        "email": email,
        "purpose": purpose
    })

    if not record:
        return False, "OTP not found or expired."

    if datetime.utcnow() > record["expires_at"]:
        otp_col.delete_one({"_id": record["_id"]})
        return False, "OTP expired."

    if record["attempts"] >= MAX_ATTEMPTS:
        return False, "Too many attempts. OTP locked."

    if record["otp"] != otp:
        otp_col.update_one(
            {"_id": record["_id"]},
            {"$inc": {"attempts": 1}}
        )
        return False, "Invalid OTP."

    # ✅ Success → delete OTP
    otp_col.delete_one({"_id": record["_id"]})
    return True, "OTP verified successfully."
```

7. recommender.py

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
def recommend_policies(age, annual_income, existing_premiums=0,
existing_policy_count=0, max_total_premium=100_000):
    """
    Fetch policies from MongoDB and recommend based on age, income, and affordability.
    Scoring:
    - Closeness to policy's ideal age and income midpoint.
    - Penalize if existing premium or policy count is high.
    Returns top 5 recommended policies with their names and scores.
    """

    from db import policies_col
    recommendations = []

    try:
        policies = policies_col.find()
    except Exception:
        return [] # If DB is unavailable

    for p in policies:
        # Safely pull fields
        min_age = p.get("min_age", 0)
        max_age = p.get("max_age", 100)
        min_inc = p.get("min_income", 0)
        max_inc = p.get("max_income", 10**12)
        premium = p.get("premium_amount", 0)

        # Eligibility check
        if not (min_age <= age <= max_age and min_inc <= annual_income <= max_inc):
            continue

        # Affordability penalty
        if existing_premiums + premium > max_total_premium:
            affordability_penalty = 0.5 # reduce score if too expensive
        else:
            affordability_penalty = 1.0

        # Policy count penalty (more existing policies slightly reduce score)
        count_penalty = max(0.7, 1.0 - 0.05 * existing_policy_count)

        # Closeness scores
        age_mid = (min_age + max_age) / 2
        inc_mid = (min_inc + max_inc) / 2

        age_range = (max_age - min_age) or 1
        inc_range = (max_inc - min_inc) or 1

        age_score = 1 - abs(age - age_mid) / age_range
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
inc_score = 1 - abs(annual_income - inc_mid) / inc_range
```

```
total_score = max(0, (age_score + inc_score) * affordability_penalty * count_penalty)
```

```
recommendations.append({
    "name": p.get("name", "Unnamed Policy"),
    "score": round(total_score, 3)
})
```

```
# Sort by score descending and return top 5
```

```
recommendations.sort(key=lambda x: x["score"], reverse=True)
```

```
return recommendations[:5]
```

8. vehicle_damage_ai.py

```
# services/vehicle_damage_ai.py
```

```
import os
```

```
from ultralytics import YOLO
```

```
# -----
```

```
# Resolve absolute model path safely
```

```
# -----
```

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
MODEL_PATH = os.path.join(BASE_DIR, "static", "models", "trained.pt")
```

```
if not os.path.exists(MODEL_PATH):
```

```
    raise FileNotFoundError(f"YOLO model not found at {MODEL_PATH}")
```

```
# Load model ONCE (important for performance)
```

```
model = YOLO(MODEL_PATH)
```

```
# -----
```

```
# Damage → Base repair cost (₹)
```

```
# -----
```

```
DAMAGE_COSTS = {
```

```
    "dent": 8000,
```

```
    "scratch": 3000,
```

```
    "crack": 15000,
```

```
    "broken_lamp": 10000,
```

```
    "shattered_glass": 25000,
```

```
    "flat_tire": 1500
```

```
}
```

```
# -----
```

```
# Vehicle-type cost multipliers
```


Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
# -----
VEHICLE_MULTIPLIER = {
    "car": 1.0,
    "bike": 0.6
}

# -----
# Severity multiplier (KEY FIX)
# -----
def severity_multiplier(damage_types_count):
    if damage_types_count >= 6:
        return 2.5, "High"
    elif damage_types_count >= 4:
        return 1.8, "Medium"
    elif damage_types_count >= 2:
        return 1.3, "Low"
    return 1.0, "Low"

# -----
# MAIN DAMAGE ASSESSMENT
# -----
def assess_vehicle_damage(image_paths, vehicle_type):
    """
    image_paths: list[str] OR single image path
    vehicle_type: car / bike
    """

    if isinstance(image_paths, str):
        image_paths = [image_paths]

    detected_counts = {} # damage → count
    confidence_log = {} # damage → max confidence (for transparency)
    breakdown = []
    total_cost = 0

    vehicle_multiplier = VEHICLE_MULTIPLIER.get(vehicle_type, 1.0)

    # 🚗 Run YOLO on all images
    for img_path in image_paths:
        results = model(img_path, conf=0.01) # ✅ REQUIRED CONF LEVEL

        for r in results:
            if not r.bboxes:
                continue
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
for box in r.bboxes:
    label = model.names.get(int(box.cls))
    if label not in DAMAGE_COSTS:
        continue

    conf = float(box.conf)

    detected_counts[label] = detected_counts.get(label, 0) + 1
    confidence_log[label] = max(confidence_log.get(label, 0), conf)

# 💰 Base cost aggregation (NO confidence reduction)
for damage, count in detected_counts.items():
    base_cost = DAMAGE_COSTS[damage]
    cost = base_cost * count
    total_cost += cost

breakdown.append({
    "damage": damage,
    "count": count,
    "base_cost": base_cost,
    "max_confidence": round(confidence_log[damage], 2),
    "subtotal": cost
})

# 🚗 Vehicle multiplier
total_cost *= vehicle_multiplier

# 🔥 Severity multiplier
sev_mult, severity = severity_multiplier(len(detected_counts))
total_cost *= sev_mult

return {
    "vehicle_type": vehicle_type,
    "images_used": len(image_paths),
    "detected_damages": detected_counts,
    "estimated_cost": int(total_cost),
    "severity": severity,
    "vehicle_multiplier": vehicle_multiplier,
    "severity_multiplier": sev_mult,
    "breakdown": breakdown,
    "model": "YOLOv8 Damage Detection"
}
```

9. base.html

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}InsureSafe{% endblock %}</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

  <!-- Custom Styles -->
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">

  <!-- Icons -->
  <script src="https://kit.fontawesome.com/a2d9d6f3a1.js"
crossorigin="anonymous"></script>
</head>

<body>

  <!-- Navbar -->
  <nav class="navbar navbar-expand-lg navbar-light bg-white shadow-sm fixed-top">
    <div class="container">
      <a class="navbar-brand fw-bold" href="{{ url_for('index') }}">Insure<span class="text-
primary">Safe</span></a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navMenu">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navMenu">
        <ul class="navbar-nav ms-auto">
          {% if session.get('user_id') %}
            {% if session.get('role') == 'admin' %}
              <li class="nav-item"><a class="nav-link" href="{{ url_for('logout') }}">Logout</a></li>
            {% else %}
              <li class="nav-item"><a class="nav-link" href="{{ url_for('policies') }}">Get
Policy</a></li>
              <li class="nav-item"><a class="nav-link" href="{{ url_for('profile') }}">Profile</a></li>
              <li class="nav-item"><a class="nav-link" href="{{ url_for('dashboard')
}}">Claims</a></li>
              <li class="nav-item"><a class="nav-link" href="{{ url_for('logout') }}">Logout</a></li>
            {% endif %}
          {% else %}
            </ul>
          {% endif %}
        </div>
      </div>
    </div>
  </nav>
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```

        <li class="nav-item"><a class="nav-link" href="{{ url_for('policies') }}">Get
Policy</a></li>
        <li class="nav-item"><a class="nav-link" href="{{ url_for('dashboard')
}}">Claims</a></li>
        <li class="nav-item"><a class="nav-link btn btn-primary text-white px-3 ms-2" href="{{
url_for('login') }}">Login</a></li>
        {% endif %}
    </ul>
</div>
</div>
</div>
</nav>

<!-- Hero Section (Optional) -->
{% block hero %}
<!-- Default Hero (optional) -->
{% endblock %}

<!-- Main Content -->
<main class="container" style="padding-top:120px; padding-bottom:50px;">
    {% with messages = get_flashed_messages() %}
    {% if messages %}
        <div class="alert alert-info">
            {% for m in messages %}{{ m }}<br>{% endfor %}
        </div>
    {% endif %}
    {% endwith %}

    {% block content %}{% endblock %}
</main>

<!-- Footer -->
<footer class="bg-dark text-white py-4 mt-5">
    <div class="container text-center small">
        © {{ current_year if current_year else '2025' }} InsureSafe — Secure Your Future with
Confidence.
    </div>
</footer>

<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script
>

</body>
</html>
```

Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

10. index.html

```
{% extends "base.html" %}

{% block hero %}
<section class="hero d-flex align-items-center">
  <div class="container">
    <div class="row align-items-center">
      <div class="col-md-6">
        <h1 class="display-4 fw-bold">Smart Insurance, Simple Choices</h1>
        <p class="lead">Compare policies, file claims, and verify documents — all in one
modern portal built for trust.</p>
        <a href="{{ url_for('get_started') }}" class="btn btn-primary btn-lg me-2">Get Started</a>
        <a href="{{ url_for('dashboard') }}" class="btn btn-outline-secondary btn-lg">File a
Claim</a>
      </div>
      <div class="col-md-6 text-center">
        
      </div>
    </div>
  </div>
</section>
{% endblock %}

{% block content %}

<!-- Features Section -->
<section class="py-5 bg-gradient">
  <div class="container">
    <div class="text-center mb-5">
      <h2 class="fw-bold">Choose Your Insurance Category</h2>
      <p class="text-muted">Flexible plans tailored to protect what matters most.</p>
    </div>
    <div class="row g-4 justify-content-center">

      <div class="col-md-4 feature-box">
        

<h5 class="mt-3">Family Protection</h5>

<p>Comprehensive security for your loved ones under one plan.</p>

</div>

<div class="col-md-4 feature-box">



<h5 class="mt-3">Health & Medical</h5>

<p>Financial safety during medical emergencies and hospitalization.</p>

</div>

<div class="col-md-4 feature-box">



<h5 class="mt-3">Car Insurance</h5>

<p>Quick claims and total road protection for your four-wheeler.</p>

</div>

<div class="col-md-4 feature-box">



<h5 class="mt-3">Bike Insurance</h5>

<p>Low-cost two-wheeler coverage with instant documentation.</p>

</div>

<div class="col-md-4 feature-box">



<h5 class="mt-3">Life Insurance</h5>

<p>Secure your family's financial future for generations.</p>

</div>

</div>

</div>

</section>

<!-- FAQ Section -->

<section class="py-5">

<div class="container">

<div class="text-center mb-5">

<h2 class="fw-bold">Frequently Asked Questions</h2>

<p class="text-muted">Answers to common queries about our policies and claims  
process.</p>

</div>

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<div class="accordion" id="faqAccordion">

 <div class="accordion-item">
 <h2 class="accordion-header" id="faqHeading1">
 <button class="accordion-button" type="button" data-bs-toggle="collapse" data-bs-
target="#faq1">
 How do I file a claim?
 </button>
 </h2>
 <div id="faq1" class="accordion-collapse collapse show" data-bs-
parent="#faqAccordion">
 <div class="accordion-body">
 You can file a claim directly through our dashboard after logging into your account.
 </div>
 </div>
 </div>

 <div class="accordion-item">
 <h2 class="accordion-header" id="faqHeading2">
 <button class="accordion-button collapsed" type="button" data-bs-toggle="collapse"
data-bs-target="#faq2">
 Can I update my documents online?
 </button>
 </h2>
 <div id="faq2" class="accordion-collapse collapse" data-bs-parent="#faqAccordion">
 <div class="accordion-body">
 Yes, documents can be uploaded securely and verified via our portal.
 </div>
 </div>
 </div>

 <div class="accordion-item">
 <h2 class="accordion-header" id="faqHeading3">
 <button class="accordion-button collapsed" type="button" data-bs-toggle="collapse"
data-bs-target="#faq3">
 Are there any hidden fees?
 </button>
 </h2>
 <div id="faq3" class="accordion-collapse collapse" data-bs-parent="#faqAccordion">
 <div class="accordion-body">
 No, all charges and policy details are transparently displayed before purchase.
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
</div>
</div>
</section>
```

```
{% endblock %}
```

## 11. dashboard.html

```
{% extends "base.html" %}
{% block content %}
```

```
{% set current_time = now if now is defined else None %}
```

```
<div class="row g-3">
```

```
<!-- ===== YOUR POLICIES ===== -->
```

```
<div class="col-lg-7">
```

```
<div class="card border-0 shadow-sm h-100">
```

```
<div class="card-body">
```

```
<h6 class="mb-3">Your Policies</h6>
```

```
{% if policies %}
```

```
<ul class="list-group list-group-flush">
```

```
{% for up in policies %}
```

```
{% set pid = up.policy._id|string %}
```

```
{% set claim = claim_map.get(pid) %}
```

```
{% set cooldown_until = cooldown_map.get(pid) if cooldown_map is defined else None
```

```
%}
```

```
{% set summary = claim_summary.get(pid, {}) if claim_summary is defined else {} %}
```

```
<li class="list-group-item d-flex justify-content-between align-items-start">
```

```
<!-- ===== POLICY INFO ===== -->
```

```
<div class="w-100">
```

```
<div class="fw-semibold">{{ up.policy.name }}</div>
```

```
<div class="small text-muted">
```

```
Status: {{ up.status }}
```

```
{% if up.doc_valid %}
```

```
· Verified
```

```
{% else %}
```

```
· Pending Verification
```

```
{% endif %}
```

```
</div>
```



# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<!-- Last claim -->
{% if claim %}
<div class="small text-muted mt-1">
 Last claim:
 {% if claim.decision == "Auto-Approve" %}
 Approved
 {% elif claim.decision == "Manual Review" %}
 Manual
 {% else %}
 Rejected
 {% endif %}

 {% if claim.risk_score is not none %}
 · Risk {{ (claim.risk_score * 100)|round(1) }}%
 {% endif %}
 · {{ claim.created_at | datetimeformat }}
</div>
{% endif %}

<!-- Mini claim summary -->
{% if summary %}
<div class="small mt-2 text-muted">
 Claims:
 ✓ {{ summary.approved or 0 }}
 · ⌚ {{ summary.pending or 0 }}
 · ✗ {{ summary.rejected or 0 }}
</div>
{% endif %}
</div>

<!-- ===== CLAIM ACTION ===== -->
<div class="ms-3 text-end">

 <!-- 🛑 BLOCK claims if policy not verified or inactive -->
 {% if not up.doc_valid or up.status != "active" %}
 <span class="badge bg-secondary"
 title="Policy verification required before claims">
 Verification Pending

 {% elif not claim %}
 <a href="{{ url_for('apply_claim', policy_id=up.policy._id) }}"
 class="btn btn-sm btn-outline-primary">
 Apply

```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% elif claim.status == "pending" %}

 Claim Pending

{% elif claim.status == "approved" %}

 Approved

{% elif claim.status == "rejected" %}
 {% if cooldown_until and current_time and cooldown_until > current_time %}

 Re-apply after {{ cooldown_until.strftime('%d %b %Y') }}

 {% else %}
 <a href="{{ url_for('apply_claim', policy_id=up.policy._id) }}"
 class="btn btn-sm btn-outline-danger">
 Re-Apply

 {% endif %}
{% endif %}

</div>

{% endfor %}

{% else %}
 <div class="text-muted">
 You haven't applied for any policies yet.
 Browse policies
 </div>
{% endif %}
</div>
</div>
</div>
</div>

<!-- ===== RECENT CLAIMS ===== -->
<h6 class="mt-4">Recent Claims</h6>

{% if claims %}
<div class="table-responsive">
 <table class="table table-sm align-middle">
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<thead>
<tr>
 <th>ID</th>
 <th>Policy</th>
 <th>Amount</th>
 <th>Risk</th>
 <th>Decision</th>
 <th>When</th>
</tr>
</thead>
<tbody>
{% for c in claims %}
<tr>
 <td>#{{ c._id }}</td>
 <td>{{ c.policy.name }}</td>
 <td>₹{{ "{:,.}".format(c.amount) }}</td>
 <td>
 {% if c.risk_score is not none %}
 {{ (c.risk_score * 100)|round(1) }}%
 {% else %}—{% endif %}
 </td>
 <td>
 {% if c.decision == "Auto-Approve" %}
 Approved
 {% elif c.decision == "Manual Review" %}
 Manual
 {% else %}
 Rejected
 {% endif %}
 </td>
 <td>{{ c.created_at | datetimeformat }}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
<div class="text-muted small">
 No claims submitted yet.
</div>
{% endif %}

{% endblock %}
```

## 12. policies.html

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% extends "base.html" %}
{% block content %}

<h4 class="mb-3">Recommended Policies for You</h4>

<!-- Filters -->
<form method="GET" class="mb-4">
 <div class="row g-2 align-items-end">

 <div class="col-md-3">
 <label class="form-label">Filter by Category:</label>
 <select name="category" class="form-select">
 <option value="">All Categories</option>
 {% for cat in ['Health','Car','Bike','Life','Family'] %}
 <option value="{{ cat }}" {% if request.args.get('category') == cat %}selected{% endif %}>
 {{ cat }}
 </option>
 {% endfor %}
 </select>
 </div>

 <div class="col-md-3">
 <label class="form-label">Min Premium:</label>
 <input type="number" name="min_amount" class="form-control"
 value="{{ request.args.get('min_amount', '') }}" placeholder="e.g. 1000">
 </div>

 <div class="col-md-3">
 <label class="form-label">Max Premium:</label>
 <input type="number" name="max_amount" class="form-control"
 value="{{ request.args.get('max_amount', '') }}" placeholder="e.g. 50000">
 </div>

 <div class="col-md-2">
 <button type="submit" class="btn btn-primary w-100">Apply</button>
 </div>
 </div>
</form>

{% set category_filter = request.args.get('category') %}
{% set min_amount = request.args.get('min_amount') %}
{% set max_amount = request.args.get('max_amount') %}

<!-- ===== RECOMMENDED ===== -->
<div class="row g-3 mb-4">
 {% for p in policies %}
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% set premium = p.premium_amount or 0 %}
{% if (p._id|string in recommended_ids)
 and (not category_filter or p.category == category_filter)
 and (not min_amount or premium >= min_amount|int)
 and (not max_amount or premium <= max_amount|int) %}

<div class="col-md-6 col-lg-4">
<div class="card border-0 shadow-sm h-100 border-success">
<div class="card-body d-flex flex-column">

<h6 class="card-title">
 {{ p.name }}
 {{ p.category }}

 {% if p._id|string in existing_policy_ids and p._id|string in recommended_ids %}
 Owned + Recommended
 {% elif p._id|string in existing_policy_ids %}
 Already Owned
 {% else %}

 Recommended (Score: {{ (scores[p._id|string]*100)|round(1) }}%)

 {% endif %}
</h6>

<p class="text-muted small flex-grow-1">{{ p.description }}</p>

<div class="mb-2">
<button class="btn btn-sm btn-outline-secondary"
 type="button"
 data-bs-toggle="collapse"
 data-bs-target="#rec{{ p._id }}">
 Requirements
</button>
<div class="collapse mt-2" id="rec{{ p._id }}">
<ul class="small mb-0">
 {% for r in (p.requirements or "").split(';') %}
 {% if r %}{{ r }}{% endif %}
 {% endfor %}

</div>
</div>

{% if p._id|string not in existing_policy_ids %}
{% if user.annual_income == 0 %}
 <button class="btn btn-secondary mt-auto" disabled
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
 title="Income must be greater than zero to apply">
 Income Required
 </button>
 {% else %}
 <a href="{{ url_for('apply_policy', pid=p._id) }}"
 class="btn btn-primary mt-auto">
 Apply

 {% endif %}
 {% else %}
 <button class="btn btn-secondary mt-auto" disabled>Already Owned</button>
 {% endif %}

</div>

<div class="card-footer small text-muted">
 Age {{ p.min_age }}–{{ p.max_age }}
 · Income ₹{{ "{:,.}".format(p.min_income or 0) }}–
 {% if p.max_income %}
 ₹{{ "{:,.}".format(p.max_income) }}
 {% else %}
 No Limit
 {% endif %}
 · Premium ₹{{ "{:,.}".format(p.premium_amount or 0) }}
</div>
</div>
</div>
{% endif %}
{% endfor %}
</div>

<!-- ===== ALL POLICIES ===== -->
<h4 class="mb-3">All Policies</h4>

<div class="row g-3">
 {% for p in policies %}
 {% set premium = p.premium_amount or 0 %}
 {% if (p._id|string not in recommended_ids)
 and (not category_filter or p.category == category_filter)
 and (not min_amount or premium >= min_amount|int)
 and (not max_amount or premium <= max_amount|int) %}

 <div class="col-md-6 col-lg-4">
 <div class="card border-0 shadow-sm h-100">
 <div class="card-body d-flex flex-column">
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<h6 class="card-title">
 {{ p.name }}
 {{ p.category }}
 {% if p._id|string in existing_policy_ids %}
 Already Owned
 {% endif %}
</h6>

<p class="text-muted small flex-grow-1">{{ p.description }}</p>

<div class="mb-2">
 <button class="btn btn-sm btn-outline-secondary"
 type="button"
 data-bs-toggle="collapse"
 data-bs-target="#all{{ p._id }}">
 Requirements
 </button>
 <div class="collapse mt-2" id="all{{ p._id }}">
 <ul class="small mb-0">
 {% for r in (p.requirements or "").split(';') %}
 {% if r %}{{ r }}{% endif %}
 {% endfor %}

 </div>
</div>

{% if p._id|string not in existing_policy_ids %}
 {% if user.annual_income == 0 %}
 <button class="btn btn-secondary mt-auto" disabled
 title="Income must be greater than zero to apply">
 Income Required
 </button>
 {% else %}
 <a href="{{ url_for('apply_policy', pid=p._id) }}"
 class="btn btn-primary mt-auto">
 Apply

 {% endif %}
{% else %}
 <button class="btn btn-secondary mt-auto" disabled>Already Owned</button>
{% endif %}

</div>

<div class="card-footer small text-muted">
 Age {{ p.min_age }}–{{ p.max_age }}
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
· Income ₹{{ "{:,.} ".format(p.min_income or 0) }}–
{% if p.max_income %}
 ₹{{ "{:,.} ".format(p.max_income) }}
{% else %}
 No Limit
{% endif %}
· Premium ₹{{ "{:,.} ".format(p.premium_amount or 0) }}
</div>
</div>
</div>
{% endif %}
{% endfor %}
</div>

{% endblock %}
```

## 13. profile.html

```
{% extends "base.html" %}
{% block content %}
<h4 class="mb-3">👤 My Profile</h4>

<div class="row g-3">
 <!-- User Info -->
 <div class="col-lg-4">
 <div class="card border-0 shadow-sm h-100">
 <div class="card-body text-center">

 <h6 class="fw-bold" id="profileName">{{ user.name }}</h6>
 <p class="text-muted small mb-1" id="profileEmail">{{ user.email }}</p>
 <p class="text-muted small mb-1" id="profilePhone">{{ user.phone or " " }}</p>
 <p class="text-muted small mb-1" id="profileAddress">{{ user.address or " " }}</p>

 <!-- 🔥 SHOW INCOME -->
 <p class="text-muted small mb-1">
 Income: ₹{{ user.annual_income or 0 }}
 </p>

 {{ 'Active' if user.is_active else 'Inactive' }}

{% if user.is_admin %}
```



# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
Admin
{% endif %}
</div>
</div>
</div>

<!-- User Policies -->
<div class="col-lg-8">
<div class="card border-0 shadow-sm h-100">
<div class="card-body">
<h6 class="mb-3">📄 My Policies</h6>
{% if policies %}
<div class="table-responsive">
<table class="table table-sm align-middle">
<thead class="table-light">
<tr>
<th>Policy</th>
<th>Status</th>
<th>Applied At</th>
</tr>
</thead>
<tbody>
{% for p in policies %}
<tr>
<td>{{ p.policy.name }}</td>
<td>
<span class="badge {{ 'bg-success' if p.status=='Approved' else 'bg-warning text-
dark' }}">
{{ p.status }}

{% if p.doc_valid %}
Verified
{% endif %}
</td>
<td>{{ p.applied_at | datetimeformat }}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
<p>No policies applied yet.</p>
{% endif %}
</div>
</div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

</div>

<!-- ===== EDIT PROFILE ===== -->

<div class="row g-3 mt-3">

<div class="col-12">

<div class="card border-0 shadow-sm">

<div class="card-body">

<h6 class="mb-3">✎ Edit Profile</h6>

<form id="editProfileForm" enctype="multipart/form-data">

<div class="mb-2">

<label class="form-label">Name</label>

<input type="text" name="name" class="form-control" value="{{ user.name }}"

required>

</div>

<div class="mb-2">

<label class="form-label">Email</label>

<input type="email" name="email" class="form-control" value="{{ user.email }}"

required>

</div>

<div class="mb-2">

<label class="form-label">Phone</label>

<input type="text" name="phone" class="form-control" value="{{ user.phone or " }}">

</div>

<div class="mb-2">

<label class="form-label">Address</label>

<textarea name="address" class="form-control">{{ user.address or " "}}</textarea>

</div>

<!-- 🔥 NEW INCOME FIELD -->

<div class="mb-2">

<label class="form-label">Annual Income (₹)</label>

<input type="number" name="annual\_income"

class="form-control"

min="0"

value="{{ user.annual\_income or 0 }}"

required>

<small class="text-muted">

Update income to become eligible for policy applications.

</small>

</div>

<div class="mb-2">

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<label class="form-label">ID Photo</label>
<input type="file" name="id_photo" class="form-control">
</div>

<button type="submit" class="btn btn-primary mt-2">
 Update Profile
</button>
</form>
</div>
</div>
</div>
</div>

<script>
document.getElementById("editProfileForm").addEventListener("submit", async function(e){
 e.preventDefault();
 let form = e.target;
 let data = new FormData(form);

 let res = await fetch("{{ url_for('edit_profile') }}", {
 method: "POST",
 body: data
 });

 let result = await res.json();

 if(result.status === "success"){
 alert(result.message);

 document.getElementById("profileName").innerText = result.user.name;
 document.getElementById("profileEmail").innerText = result.user.email;
 document.getElementById("profilePhone").innerText = result.user.phone || "";
 document.getElementById("profileAddress").innerText = result.user.address || "";
 document.getElementById("profileIncome").innerText = result.user.annual_income || 0;

 if(result.user.id_photo_path){
 document.getElementById("profilePhoto").src =
 "{{ url_for('uploaded_file', filename=) }}" +
 result.user.id_photo_path + "?t=" + Date.now();
 }
 } else {
 alert(result.message);
 }
});
</script>
```

# Title: Insuresafe

Language of Work: Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% endblock %}
```

## 14. admin.html

```
{% extends "base.html" %}
{% block content %}
<h4 class="mb-3"> ⚙ Admin Panel</h4>

<div class="row g-3">
<!-- ===== STATS (VERTICAL) ===== -->
<div class="col-lg-4 col-md-12">
 <div class="card border-0 shadow-sm rounded-4 h-100">
 <div class="card-body d-flex flex-column justify-content-between px-4">

 <!-- USERS -->
 <div class="text-center py-4 border-bottom">
 <div class="stat-label"> 👤 Users</div>
 <div class="stat-number">{{ counts.n_users }}</div>
 </div>

 <!-- POLICIES -->
 <div class="text-center py-4 border-bottom">
 <div class="stat-label"> 📄 Policies</div>
 <div class="stat-number">{{ counts.n_up }}</div>
 </div>

 <!-- CLAIMS -->
 <div class="text-center py-4">
 <div class="stat-label"> 📄 Claims</div>
 <div class="stat-number">{{ counts.n_c }}</div>
 </div>

 </div>
 </div>

 <!-- ===== USERS TABLE ===== -->
 <div class="col-lg-8 col-md-12">
 <div class="card border-0 shadow-sm rounded-3 h-100">
 <div class="card-body">
 <div class="d-flex justify-content-between align-items-center mb-3">
 <h6 class="mb-0"> 👥 Registered Users</h6>
 <input type="text" id="userSearch"
 class="form-control form-control-sm w-50"
 placeholder="🔍 Search users...">
 </div>
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

</div>

```
<div class="table-responsive">
```

```
<table class="table table-sm align-middle table-hover" id="usersTable">
```

```
<thead class="table-light">
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Name</th>
```

```
<th>Email</th>
```

```
<th>Role</th>
```

```
<th>Email Status</th>
```

```
<th>Account</th>
```

```
<th class="text-center">Action</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
{% for u in users %}
```

```
<tr>
```

```
<td>{{ u._id|string }}</td>
```

```
<td>{{ u.name }}</td>
```

```
<!-- EMAIL + VERIFIED BADGE -->
```

```
<td>
```

```
 {{ u.email }}
```

```
</td>
```

```
<!-- ROLE -->
```

```
<td>
```

```
 {% if u.is_admin %}
```

```
 Admin
```

```
 {% else %}
```

```
 User
```

```
 {% endif %}
```

```
</td>
```

```
<!-- EMAIL VERIFIED -->
```

```
<td>
```

```
 {% if u.email_verified %}
```

```
 ✔ Verified
```

```
 {% else %}
```

```
 ⚠ Not Verified
```

```
 {% endif %}
```

```
</td>
```

```
<!-- ACCOUNT STATUS -->
```

```
<td>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```

 {{ 'Active' if u.is_active else 'Inactive' }}

</td>

<!-- ACTIONS -->
<td class="text-center">
 {% if not u.is_admin %}
 <div class="d-flex gap-1 justify-content-center">
 <a href="{{ url_for('admin_toggle', uid=u._id|string) }}"
 class="btn btn-sm btn-outline-primary">
 Toggle

 {% if not u.email_verified %}
 <a href="{{ url_for('admin_verify_email', uid=u._id|string) }}"
 class="btn btn-sm btn-outline-success">
 Verify

 {% endif %}
 </div>
 {% else %}
 —
 {% endif %}
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>

</div>
</div>
</div>
</div>
```

```
<!-- ===== CLAIMS TABLE ===== -->
<div class="card border-0 shadow-sm mt-4">
 <div class="card-header bg-dark text-white"> 📄 Claims Overview</div>
 <div class="card-body table-responsive">
 <table class="table table-hover align-middle">
 <thead>
 <tr>
 <th>ID</th><th>Policy</th><th>Type</th>
 <th>Amount</th><th>Risk</th><th>Decision</th><th>Date</th>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
</tr>
</thead>
<tbody>
{% for c in claims %}
<tr>
<td>#{{ c._id }}</td>
<td>{{ c.policy.name }}</td>
<td>
{% if c.claim_type == "vehicle" %}
 🚗 Vehicle
{% else %}
 📄 Document
{% endif %}
</td>
<td>₹{{ "{:,.}".format(c.amount) }}</td>
<td>{{ (c.risk_score * 100)|round(1) if c.risk_score else "—" }}%</td>
<td>
{% if c.decision == "Auto-Approve" %}
Approved
{% elif c.decision == "Manual Review" %}
Manual
{% else %}
Rejected
{% endif %}
</td>
<td>{{ c.created_at | datetimeformat }}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>

<!-- ===== POLICY APPLICATIONS ===== -->
<div class="card border-0 shadow-sm rounded-3 mt-4">
<div class="card-header bg-warning text-dark fw-bold">
📄 Policy Applications (Document Review)
</div>

<div class="card-body table-responsive">
<table class="table table-hover align-middle">
<thead>
<tr>
<th>User</th><th>Policy</th><th>Status</th>
<th>Documents</th><th>Action</th>
</tr>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
</thead>
<tbody>
{% for up in user_policies %}
<tr>
<td>{{ up.user.name }}</td>
<td>{{ up.policy.name }}</td>
<td>
{% if up.status == "active" %}
Active
{% else %}
Pending
{% endif %}
</td>

<td>
{% if up.uploaded_docs %}
<button class="btn btn-sm btn-outline-primary"
data-bs-toggle="modal"
data-bs-target="#docsModal{{ up._id }}">
📄 View Docs
</button>
{% else %}
—
{% endif %}
</td>

<td>
{% if not up.doc_valid %}
<a href="{{ url_for('admin_verify_policy', user_policy_id=up._id|string) }}"
class="btn btn-sm btn-success">Approve
{% else %}—{% endif %}
</td>
</tr>

<!-- MODAL -->
<div class="modal fade" id="docsModal{{ up._id }}" tabindex="-1">
<div class="modal-dialog modal-lg modal-dialog-scrollable">
<div class="modal-content">
<div class="modal-header">
<h5 class="modal-title">📁 Uploaded Documents</h5>
<button class="btn-close" data-bs-dismiss="modal"></button>
</div>
<div class="modal-body">
{% for key, path in up.uploaded_docs.items() %}
<div class="d-flex justify-content-between py-2 border-bottom">
{{ key.replace('_', ' ').title() }}
```



# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<a href="{{ url_for('admin_files', filename=path) }}"
 target="_blank"
 class="btn btn-sm btn-outline-success">
 View

</div>
</div>
{% endfor %}
</div>
</div>
</div>
</div>
{% endfor %}
</tbody>
</table>
</div>
</div>

<!-- ===== POLICIES TABLE ===== -->
<div class="card border-0 shadow-sm rounded-3 mt-4">
 <div class="card-header bg-primary text-white fw-bold d-flex justify-content-between align-items-center">
  Manage Policies
 <button class="btn btn-light btn-sm" data-bs-toggle="modal"
 data-bs-target="#addPolicyModal"> + Add Policy</button>
 </div>
 <div class="card-body table-responsive">
 <table class="table table-hover">
 <thead>
 <tr>
 <th>Name</th>
 <th>Category</th>
 <th>Premium</th>
 <th>Action</th>
 </tr>
 </thead>
 <tbody>
 {% for p in policy_defs %}
 <tr>
 <td>{{ p.name }}</td>
 <td>{{ p.category }}</td>
 <td>₹{{ "{:,.}".format(p.premium_amount) }}</td>
 <td>
 <a href="{{ url_for('admin_delete_policy', policy_id=p._id|string) }}"
 class="btn btn-sm btn-danger">Delete
 </td>
 </tr>
 </tbody>
 </table>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% endfor %}
</tbody>
</table>
</div>
</div>

<script>
document.getElementById("userSearch").addEventListener("keyup", function() {
 let filter = this.value.toLowerCase();
 document.querySelectorAll("#usersTable tbody tr").forEach(row => {
 row.style.display = row.innerText.toLowerCase().includes(filter) ? "" : "none";
 });
});
</script>
{% endblock %}
```

## 15. admin\_policy\_docs.html

```
{% extends "base.html" %}
{% block content %}

<h4 class="mb-3">📁 Uploaded Documents</h4>

{% if docs %}
<div class="list-group">
 {% for key, path in docs.items() %}
 <a href="{{ url_for('admin_files', filename=path) }}"
 target="_blank"
 class="list-group-item list-group-item-action">
 📄 {{ key.replace('_', ' ').title() }}

 {% endfor %}
</div>
{% else %}
<p class="text-muted">No documents uploaded.</p>
{% endif %}

 ⬅ Back to Admin Panel

{% endblock %}
```

## 16. apply\_policy.html

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% extends "base.html" %}
{% block content %}
<div class="container my-4">
 <div class="row">

 <!-- ===== POLICY INFO ===== -->
 <div class="col-lg-4">
 <div class="card shadow-sm border-0 mb-3">
 <div class="card-body">
 <h5 class="card-title">Policy: {{ policy.name }}</h5>
 <p class="text-muted small">{{ policy.description }}</p>
 <div class="mt-3">
 Category: {{ policy.category }}

 Premium: ₹{{ policy.premium_amount }}

 Duration: {{ policy.duration_years }} year(s)
 </div>
 </div>
 </div>
 </div>

 <!-- ===== APPLICATION FORM ===== -->
 <div class="col-lg-8">
 <div class="card shadow-sm border-0">
 <div class="card-body">
 <h5 class="mb-3">Apply for this Policy</h5>

 <form method="post" enctype="multipart/form-data">

 <!-- ===== PERSONAL INFO ===== -->
 <h6 class="text-primary mb-2">Personal Details</h6>
 <div class="row g-3 mb-2">

 <div class="col-md-6">
 <label class="form-label">Full Name</label>
 <input type="text"
 name="name"
 class="form-control"
 value="{{ user.name }}"
 required>
 </div>

 <div class="col-md-3">
 <label class="form-label">Date of Birth</label>
 <input type="date"
 name="dob"
 class="form-control">
 </div>
 </div>
 </form>
 </div>
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
 required>
 </div>

 <div class="col-md-3">
 <label class="form-label">Gender</label>
 <select name="gender" class="form-select" required>
 <option value="">Select</option>
 <option value="Male" {% if user.gender == "Male" %}>selected{% endif
%}>Male</option>
 <option value="Female" {% if user.gender == "Female" %}>selected{% endif
%}>Female</option>
 <option value="Other" {% if user.gender == "Other" %}>selected{% endif
%}>Other</option>
 </select>
 </div>

</div>

<!-- ===== CONTACT INFO ===== -->
<h6 class="text-primary mb-2 mt-3">Contact Details</h6>
<div class="row g-3 mb-2">

 <div class="col-md-6">
 <label class="form-label">Email</label>
 <input type="email"
 name="email"
 class="form-control"
 value="{{ user.email }}"
 required>
 </div>

 <div class="col-md-6">
 <label class="form-label">Phone</label>
 <input type="text"
 name="phone"
 class="form-control"
 value="{{ user.phone or '' }}"
 required>
 </div>

</div>

<!-- ===== ADDRESS ===== -->
<h6 class="text-primary mb-2 mt-3">Address</h6>
<div class="row g-3 mb-2">
 <div class="col-md-12">
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<label class="form-label">Full Address</label>
<textarea name="address"
 class="form-control"
 rows="2"
 required>{{ user.address or " " }}</textarea>
</div>
</div>

<!-- ===== REQUIRED DOCUMENTS ===== -->
<h6 class="text-primary mb-2 mt-3">Required Documents</h6>
<div class="row g-3 mb-2">

 {% for key, label in requirements.items() %}
 <div class="col-md-6 {% if key == 'family_details' %}col-md-12{% endif %}">
 <label class="form-label">{{ label }}</label>
 <input type="file"
 name="{{ key }}"
 class="form-control"
 accept="image/*,.pdf"
 required>
 </div>
 {% endfor %}

</div>

<button class="btn btn-success w-100 mt-3">
 Submit Application
</button>
</form>

{% if result %}
<div class="alert alert-info mt-3">
 Verification Result: {{ result.is_valid }}
</div>
{% endif %}

</div>
</div>
</div>

</div>
</div>
{% endblock %}
```

## 17. apply\_claim.html

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% extends "base.html" %}
{% block content %}

<div class="row g-4">

 <!-- ===== CLAIM TYPE SELECT ===== -->
 <div class="col-12">
 <div class="btn-group w-100 mb-3">
 <button class="btn btn-outline-primary {% if claim_type == 'vehicle' %}active{% endif %}"
 onclick="location.href='?type=vehicle'">
 🚗 Vehicle Claim
 </button>
 <button class="btn btn-outline-secondary {% if claim_type == 'document' %}active{%
endif %}"
 onclick="location.href='?type=document'">
 📄 Document Claim
 </button>
 </div>
 </div>

 <!-- ===== VEHICLE CLAIM ===== -->
 <!-- ===== -->
 {% if claim_type == "vehicle" %}
 <div class="col-lg-5">
 <div class="card shadow-sm border-0">
 <div class="card-body">
 <h5 class="card-title"> 🚗 Vehicle Damage Assessment</h5>

 {% if not report or not report.estimated_damage %}
 <form method="post" enctype="multipart/form-data">
 <input type="hidden" name="claim_type" value="vehicle">
 <input type="hidden" name="stage" value="estimate">

 <div class="mb-3">
 <label class="form-label">Upload Damage Images</label>
 <input type="file"
 name="claim_files"
 class="form-control"
 accept="image/*"
 multiple
 required>
 <small class="text-muted">
 Upload 1–5 clear images from different angles
 </small>
 </div>
 </form>
 {% else %}
 <div class="text-center">
 <h3>🚗 Vehicle Damage Assessment</h3>
 <p>📄 Document Claim</p>
 </div>
 {% endif %}
 </div>
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<button class="btn btn-primary w-100"
 onclick="this.disabled=true; this.form.submit();">
 Analyze Damage
</button>
</form>
{% endif %}
</div>
</div>
</div>

<div class="col-lg-7">
<h5 class="mb-3">Assessment</h5>

{% if report and report.estimated_damage %}
<div class="card shadow-sm mb-3">
<div class="card-body">
<h6 class="fw-bold">AI Estimated Damage Cost</h6>
<h3 class="text-success">₹{{ "{:,.} ".format(report.estimated_damage) }}</h3>
<div class="text-muted small">
 Based on {{ report.signals.images_used }} image(s) (averaged)
</div>
</div>
</div>
</div>

<div class="card shadow-sm">
<div class="card-body">
<h6 class="fw-bold">Your Claim Amount</h6>

<form method="post">
<input type="hidden" name="claim_type" value="vehicle">
<input type="hidden" name="stage" value="bargain">
<input type="hidden" name="ai_estimate" value="{{ report.estimated_damage }}">

<input type="number"
 name="claim_amount"
 class="form-control mb-2"
 value="{{ report.estimated_damage }}"
 min="{{ (report.estimated_damage * 0.7)|int }}"
 max="{{ (report.estimated_damage * 1.5)|int }}"
 required>

<button class="btn btn-success w-100"
 onclick="this.disabled=true; this.form.submit();">
 Submit Vehicle Claim
</button>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
</form>
</div>
</div>

{% elif report and report.decision %}
<div class="card shadow-sm">
<div class="card-body">
<h5 class="fw-bold">Final Decision</h5>

<span class="badge
{% if report.decision == 'Auto-Approve' %}bg-success
{% elif report.decision == 'Manual Review' %}bg-warning text-dark
{% else %}bg-danger{% endif %}">
{{ report.decision }}

</div>
</div>
{% endif %}
</div>
{% endif %}

<!-- ===== -->
<!-- ===== DOCUMENT CLAIM ===== -->
<!-- ===== -->
{% if claim_type == "document" %}
<div class="col-lg-6">
<div class="card shadow-sm border-0">
<div class="card-body">
<h5 class="card-title"> 📄 Document-based Claim</h5>

<form method="post" enctype="multipart/form-data">
<input type="hidden" name="claim_type" value="document">

<div class="mb-3">
<label class="form-label">Upload Claim Documents</label>
<input type="file"
name="claim_files"
class="form-control"
multiple
required>
<small class="text-muted">
Bills, reports, prescriptions, discharge summary
</small>
</div>

<div class="mb-3">
```



# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<label class="form-label">Claim Amount</label>
<input type="number" name="claim_amount" class="form-control" required>
</div>

<button class="btn btn-primary w-100"
 onclick="this.disabled=true; this.form.submit();">
 Submit Document Claim
</button>
</form>
</div>
</div>
</div>

<div class="col-lg-6">
 {% if report %}
 <div class="card shadow-sm">
 <div class="card-body">
 <h6 class="fw-bold">AI Decision</h6>

 <span class="badge
 {% if report.decision == 'Auto-Approve' %}bg-success
 {% elif report.decision == 'Manual Review' %}bg-warning text-dark
 {% else %}bg-danger{% endif %}">
 {{ report.decision }}

 <div class="mt-2 text-muted">
 Risk Score: {{ (report.risk_score * 100)|round(1) }}%
 </div>

 <pre class="small mt-2 bg-light p-2 rounded">
 {{ report.signals | tojson(indent=2) }}
 </pre>
 </div>
 </div>
 {% endif %}
</div>
{% endif %}

</div>

{% endblock %}
```

## 18. claims\_ai.html

```
{% extends "base.html" %}
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% block content %}
<div class="row g-4">
 <div class="col-lg-5">
 <div class="card shadow-sm border-0">
 <div class="card-body">
 <h5 class="card-title">AI-Assisted Claims Processing</h5>

 <form method="post" enctype="multipart/form-data">

 <!-- POLICY VEHICLE TYPE (READ-ONLY) -->
 {% if policy.category in ["Car", "Bike"] %}
 <div class="mb-3">
 <label class="form-label">Vehicle Type (From Policy)</label>
 <input type="text"
 class="form-control"
 value="{{ policy.category }}"
 readonly>
 </div>
 {% endif %}

 <!-- CLAIM TYPE -->
 <div class="mb-3">
 <label class="form-label">Claim Type</label>
 <select name="claim_type"
 id="claimType"
 class="form-select"
 required>
 <option value="">Select Claim Type</option>

 {% if policy.category in ["Car", "Bike"] %}
 <option value="vehicle"> 🚗 Vehicle Damage Claim</option>
 {% endif %}

 <option value="document"> 📄 Document-based Claim</option>
 </select>
 </div>

 <!-- CLAIM AMOUNT -->
 <div class="mb-3">
 <label class="form-label">Claim Amount (₹)</label>
 <input type="number"
 name="claim_amount"
 class="form-control"
 required>
 </div>
 </form>
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<!-- SUM INSURED -->
<div class="mb-3">
 <label class="form-label">Policy Sum Insured (₹)</label>
 <input type="number"
 name="sum_insured"
 class="form-control"
 value="{{ policy.sum_insured }}"
 readonly>
</div>

<!-- FILE UPLOAD -->
<div class="mb-3">
 <label class="form-label">Upload Claim Files</label>
 <input type="file"
 name="claim_files"
 id="claimFiles"
 class="form-control"
 multiple
 accept="image/*,.pdf">
 <small id="fileHint" class="text-muted">
 Select claim type to see allowed uploads.
 </small>
</div>

<button class="btn btn-primary w-100">
 Assess Claim
</button>
</form>
</div>
</div>
</div>

<!-- ===== ASSESSMENT ===== -->
<div class="col-lg-7">
 <h5 class="mb-3">Assessment</h5>

 {% if report %}
 <div class="card border-0 shadow-sm mb-3">
 <div class="card-body">
 <div class="d-flex justify-content-between align-items-center">
 <div>
 <div class="fw-bold">
 Decision: {{ report.decision }}
 </div>
 <div class="text-muted">
 Fraud Risk: {{ (report.risk_score * 100)|round(1) }}%
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```

 </div>
 </div>
 <span class="badge
 {% if report.decision == 'Auto-Approve' %}
 bg-success
 {% elif report.decision == 'Manual Review' %}
 bg-warning text-dark
 {% else %}
 bg-danger
 {% endif %}
 p-2">
 {{ report.decision }}

</div>
</div>
</div>

<div class="card border-0 shadow-sm">
 <div class="card-body">
 <h6 class="mb-2">Signals</h6>
 <pre class="small mb-0">
{{ report.signals | tojson(indent=2) }}
 </pre>
 </div>
</div>
{% else %}
 <div class="text-muted">
 Enter details and upload files to get an assessment.
 </div>
{% endif %}
</div>
</div>

<!-- ===== UI LOGIC ===== -->
<script>
const claimType = document.getElementById("claimType");
const fileInput = document.getElementById("claimFiles");
const hint = document.getElementById("fileHint");

claimType.addEventListener("change", function () {
 if (this.value === "vehicle") {
 fileInput.accept = "image/*";
 hint.innerText =
 " 🚗 Upload ONLY vehicle damage images (no PDFs).";
 } else if (this.value === "document") {
 fileInput.accept = "image/*,.pdf";
 }
});

```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
hint.innerText =
 "📄 Upload policy documents / reports only.";
} else {
 fileInput.accept = "image/*,.pdf";
 hint.innerText =
 "Select claim type to see allowed uploads.";
}
});
</script>

{% endblock %}
```

## 19. document\_verification.html

```
{% extends "base.html" %}
{% block content %}
<div class="row g-4">
 <div class="col-lg-5">
 <div class="card shadow-sm border-0">
 <div class="card-body">
 <h5 class="card-title">Document Verification (OCR + Classification)</h5>
 <form method="post" enctype="multipart/form-data">
 <div class="mb-3">
 <label class="form-label">Full Name (expected)</label>
 <input type="text" name="name" class="form-control" required>
 </div>
 <div class="mb-3">
 <label class="form-label">Date of Birth (expected)</label>
 <input type="date" name="dob" class="form-control" required>
 </div>
 <div class="mb-3">
 <label class="form-label">ID Number (expected)</label>
 <input type="text" name="id_number" class="form-control" required>
 </div>
 <div class="mb-3">
 <label class="form-label">Upload Document Image</label>
 <input type="file" name="document" class="form-control" accept="image/*,.pdf"
required>
 </div>
 <button class="btn btn-primary w-100">Verify Document</button>
 </form>
 </div>
 </div>
 </div>
 <div class="col-lg-7">
 <h5 class="mb-3">Result</h5>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% if result %}
<div class="card border-0 shadow-sm">
 <div class="card-body">
 <div class="mb-2">{{ result.document_type
}}</div>
 <pre class="small mb-3">{{ result.extracted | tojson(indent=2) }}</pre>
 <div>
 {% if result.is_valid %}
 Validated
 {% else %}
 Mismatch
 {% endif %}
 </div>
 </div>
</div>
{% else %}
 <div class="text-muted">Provide expected details and upload a document image to
verify.</div>
{% endif %}
</div>
</div>
{% endblock %}
```

## 20. face\_verification.html

```
{% extends "base.html" %}
{% block content %}
<div class="row g-4">
 <div class="col-lg-5">
 <div class="card shadow-sm border-0">
 <div class="card-body">
 <h5 class="card-title">Facial Verification (CNN)</h5>
 <form method="post" enctype="multipart/form-data">
 <div class="mb-3">
 <label class="form-label">Upload ID Photo</label>
 <input type="file" name="id_photo" class="form-control" accept="image/*" required>
 </div>
 <div class="mb-3">
 <label class="form-label">Upload Live Selfie</label>
 <input type="file" name="selfie" class="form-control" accept="image/*" required>
 </div>
 <button class="btn btn-primary w-100">Verify Face</button>
 </form>
 </div>
 </div>
 </div>
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<div class="col-lg-7">
 <h5 class="mb-3">Result</h5>
 {% if result %}
 <div class="card border-0 shadow-sm">
 <div class="card-body">
 <div class="d-flex align-items-center justify-content-between">
 <div>
 <div class="fw-bold">Match: {{ 'Yes' if result.match else 'No' }}</div>
 <div class="text-muted small">Similarity: {{ (result.similarity * 100)|round(1) }}%</div>
 </div>
 {{ 'Approved'
if result.match else 'Rejected' }}
 </div>
 </div>
 </div>
 {% else %}
 <div class="text-muted">Upload images to see the verification result.</div>
 {% endif %}
</div>
</div>
{% endblock %}
```

## 21. auth\_login.html

```
{% extends "base.html" %}

{% block content %}
<div class="row justify-content-center">
 <div class="col-md-6">
 <div class="card card-modern shadow-sm border-0">
 <div class="card-body">
 <h5 class="card-title mb-4">Login</h5>

 <form method="post">
 <div class="mb-3">
 <label class="form-label" for="email">Email</label>
 <input type="email" id="email" name="email" class="form-control" required>
 </div>

 <div class="mb-3">
 <label class="form-label" for="password">Password</label>
 <input type="password" id="password" name="password" class="form-control"
required>
 </div>

 <div class="d-flex justify-content-between align-items-center mb-3">
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
Forgot password?
<button type="submit" class="btn btn-primary">Login</button>
</div>
</form>

<div class="text-center mt-3">
 New here?
 Create an account
</div>

</div>
</div>
</div>
</div>
{% endblock %}
```

## 22. auth\_signup.html

```
{% extends "base.html" %}
{% block content %}
<div class="row justify-content-center">
 <div class="col-md-8 col-lg-6">
 <div class="card shadow-sm border-0">
 <div class="card-body">
 <h5 class="card-title mb-3">Sign Up</h5>

 <form method="post" enctype="multipart/form-data">
 <!-- 🗝 STEP CONTROL -->
 <input type="hidden" name="step" value="{{ 'verify_otp' if otp_sent else 'send_otp' }}">

 <!-- FULL NAME -->
 <div class="mb-3">
 <label class="form-label">Full Name</label>
 <input type="text" name="name" class="form-control"
 value="{{ request.form.name }}" required {% if otp_sent %}readonly{% endif %}>
 </div>

 <!-- EMAIL -->
 <div class="mb-3">
 <label class="form-label">Email</label>
 <input type="email" name="email" class="form-control"
 value="{{ email or request.form.email }}" required {% if otp_sent %}readonly{% endif %}>
 </div>

 <!-- PASSWORD -->
```



# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<div class="mb-3">
 <label class="form-label">Password</label>
 <input type="password" name="password" class="form-control"
 required {% if otp_sent %}readonly{% endif %}>
</div>

<!-- PHONE -->
<div class="mb-3">
 <label class="form-label">Phone Number</label>
 <input type="tel" name="phone" class="form-control"
 pattern="[0-9]{10}" placeholder="10-digit number"
 value="{{ request.form.phone }}" required {% if otp_sent %}readonly{% endif %}>
</div>

<!-- ADDRESS -->
<div class="mb-3">
 <label class="form-label">Address</label>
 <textarea name="address" class="form-control" rows="2"
 {% if otp_sent %}readonly{% endif %}>{{ request.form.address }}</textarea>
</div>

<!-- AGE -->
<div class="mb-3">
 <label class="form-label">Age</label>
 <input type="number" name="age" class="form-control"
 min="18" max="100"
 value="{{ request.form.age }}" required {% if otp_sent %}readonly{% endif %}>
</div>

<!-- INCOME -->
<div class="mb-3">
 <label class="form-label">Annual Income (₹)</label>
 <input type="number" name="annual_income" class="form-control"
 min="0" step="10000"
 value="{{ request.form.annual_income }}" required {% if otp_sent %}readonly{%
endif %}>
</div>

<!-- ID PHOTO (ONLY STEP 1) -->
{% if not otp_sent %}
<div class="mb-3">
 <label class="form-label">Upload ID Photo (used for face auth)</label>
 <input type="file" name="id_photo" class="form-control" accept="image/*" required>
</div>
{% endif %}
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
<!-- 🛑 OTP INPUT (STEP 2) -->
{% if otp_sent %}
<div class="mb-3">
 <label class="form-label">Enter OTP</label>
 <input type="text" name="otp" class="form-control" placeholder="6-digit OTP"
required>

<!-- ⌚ TIMER -->
<small class="text-muted d-block mt-1">
 OTP expires in 300 seconds
</small>

<!-- 🔄 RESEND -->
<button type="button"
 id="resendBtn"
 class="btn btn-link p-0 mt-1"
 onclick="resendOtp()"
 disabled>
 Resend OTP (60s)
</button>
</div>
{% endif %}

<!-- BUTTON -->
<button class="btn btn-primary w-100">
 {{ "Verify OTP & Create Account" if otp_sent else "Send OTP" }}
</button>
</form>

<div class="text-center mt-3">
 Already have an account? Login
</div>

</div>
</div>
</div>
</div>

{% if otp_sent %}
<script>
let otpTime = 300; // 5 minutes
let resendTime = 60; // 60 seconds cooldown

const timerEl = document.getElementById("otp-timer");
const resendBtn = document.getElementById("resendBtn");
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
// OTP countdown
const otpInterval = setInterval(() => {
 otpTime--;
 if (timerEl) timerEl.innerText = otpTime;

 if (otpTime <= 0) {
 clearInterval(otpInterval);
 alert("OTP expired. Please resend.");
 resendBtn.disabled = false;
 resendBtn.innerText = "Resend OTP";
 }
}, 1000);

// Resend cooldown
const resendInterval = setInterval(() => {
 resendTime--;
 resendBtn.innerText = `Resend OTP (${resendTime}s)`;

 if (resendTime <= 0) {
 resendBtn.disabled = false;
 resendBtn.innerText = "Resend OTP";
 clearInterval(resendInterval);
 }
}, 1000);

function resendOtp() {
 resendBtn.disabled = true;
 resendBtn.innerText = "Sending...";

 fetch("/auth/resend-otp", { method: "POST" })
 .then(res => res.json())
 .then(data => {
 alert(data.msg);
 resendTime = 60;
 });
}
</script>
{% endif %}
{% endblock %}
```

## 23. auth\_face.html

```
{% extends "base.html" %}
{% block content %}

<style>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
#loader {
 display: none;
 text-align: center;
 margin-top: 15px;
}
.spinner {
 border: 4px solid #f3f3f3;
 border-top: 4px solid #007bff;
 border-radius: 50%;
 width: 40px;
 height: 40px;
 animation: spin 1s linear infinite;
 margin: auto;
}
@keyframes spin {
 0% { transform: rotate(0deg); }
 100% { transform: rotate(360deg); }
}
</style>

<script defer src="https://cdn.jsdelivr.net/npm/face-api.js@0.22.2/dist/face-api.min.js"></script>

<div class="row justify-content-center">
 <div class="col-md-6">
 <div class="card shadow-sm border-0 mt-4">
 <div class="card-body">
 <h5 class="card-title mb-3">Face Verification</h5>
 <p class="text-muted small">Align your face within the box and click capture.</p>

 <!-- Live Camera View -->
 <div id="cameraContainer">
 <div id="cameraWrapper" style="position: relative;">
 <video id="camera" autoplay playsinline width="100%" style="border-radius:10px;"></video>
 <canvas id="faceOverlay" style="position:absolute; top:0; left:0;"></canvas>
 </div>
 <button id="captureBtn" class="btn btn-primary w-100 mt-3">📷 Capture
Photo</button>
 </div>

 <!-- Preview After Capture -->
 <div id="previewContainer" style="display:none;">

 <div class="d-flex gap-2">
 <button id="retakeBtn" class="btn btn-secondary w-50">🔄 Retake</button>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
 <button id="submitBtn" class="btn btn-success w-50">✅ Verify</button>
 </div>
</div>

<!-- Loader -->
<div id="loader">
 <div class="spinner"></div>
 <p class="mt-2">Verifying face, please wait...</p>
</div>

<!-- Hidden Form -->
<form id="uploadForm" method="POST" style="display:none;">
 <input type="hidden" name="captured_image" id="captured_image">
</form>

 <canvas id="snapshot" style="display:none;"></canvas>
</div>
</div>
</div>
</div>

<script>
const video = document.getElementById("camera");
const overlay = document.getElementById("faceOverlay");
const canvas = document.getElementById("snapshot");
const previewImage = document.getElementById("previewImage");

const cameraContainer = document.getElementById("cameraContainer");
const previewContainer = document.getElementById("previewContainer");
const loader = document.getElementById("loader");

const captureBtn = document.getElementById("captureBtn");
const retakeBtn = document.getElementById("retakeBtn");
const submitBtn = document.getElementById("submitBtn");
const uploadForm = document.getElementById("uploadForm");
const inputImage = document.getElementById("captured_image");

async function loadModels() {
 try {
 await faceapi.nets.tinyFaceDetector.loadFromUri("/static/models/");
 } catch (err) {
 console.log("⚠️ model load error:", err);
 }
}
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
async function startCamera() {
 await loadModels();

 navigator.mediaDevices.getUserMedia({ video: true })
 .then(stream => {
 video.srcObject = stream;

 video.onloadedmetadata = () => {
 video.play();
 detectFace();
 };
 })
 .catch(err => {
 alert("Camera access denied or unavailable!");
 console.log(err);
 });
}

async function detectFace() {
 overlay.width = video.videoWidth;
 overlay.height = video.videoHeight;
 const ctx = overlay.getContext("2d");

 async function loop() {
 const detections = await faceapi.detectAllFaces(
 video,
 new faceapi.TinyFaceDetectorOptions({ inputSize: 320, scoreThreshold: 0.5 })
);

 ctx.clearRect(0, 0, overlay.width, overlay.height);

 detections.forEach(det => {
 const { x, y, width, height } = det.box;
 ctx.lineWidth = 3;
 ctx.strokeStyle = "lime";
 ctx.strokeRect(x, y, width, height);
 });

 requestAnimationFrame(loop);
 }
 loop();
}

captureBtn.addEventListener("click", () => {
 canvas.width = video.videoWidth;
 canvas.height = video.videoHeight;
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
canvas.getContext("2d").drawImage(video, 0, 0);

previewImage.src = canvas.toDataURL("image/jpeg", 0.35); // compress more

cameraContainer.style.display = "none";
previewContainer.style.display = "block";
});

retakeBtn.addEventListener("click", () => {
 previewContainer.style.display = "none";
 cameraContainer.style.display = "block";
});

submitBtn.addEventListener("click", () => {
 loader.style.display = "block";
 previewContainer.style.display = "none";

 inputImage.value = previewImage.src;
 uploadForm.submit();
});

startCamera();
</script>

{% endblock %}
```

## 24. auth\_forgot.html

```
{% extends "base.html" %}
{% block content %}

<div class="container mt-5" style="max-width: 500px;">
 <div class="card shadow-sm">
 <div class="card-body">

 <h4 class="card-title mb-3 text-center">
 {% if reset %}
 🗝️ Reset Password
 {% elif otp_sent %}
 🔑 Verify OTP
 {% else %}
 🗝️ Forgot Password
 {% endif %}
 </h4>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
 {% for category, message in messages %}
 <div class="alert alert-{{ category }} alert-dismissible fade show">
 {{ message }}
 <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
 </div>
 {% endfor %}
{% endif %}
{% endwith %}
```

```
<!-- ===== STEP 1: EMAIL ===== -->
{% if not otp_sent and not reset %}
<form method="POST">
 <input type="hidden" name="step" value="send_otp">

 <div class="mb-3">
 <label class="form-label">Registered Email</label>
 <input type="email" name="email" class="form-control" required>
 </div>

 <button class="btn btn-primary w-100">
 Send OTP
 </button>
</form>
{% endif %}
```

```
<!-- ===== STEP 2: OTP ===== -->
{% if otp_sent %}
<form method="POST">
 <input type="hidden" name="step" value="verify_otp">
 <input type="hidden" name="email" value="{{ email }}">

 <div class="mb-3">
 <label class="form-label">Enter OTP</label>
 <input
 type="text"
 name="otp"
 class="form-control"
 placeholder="6-digit OTP"
 pattern="[0-9]{6}"
 required
 >
 <small class="text-muted">
 OTP valid for 300 seconds
 </small>
 </div>
</form>
```



# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
</small>
</div>

<button class="btn btn-success w-100">
 Verify OTP
</button>
</form>

<!-- RESEND OTP -->
<form method="POST" class="mt-2">
 <input type="hidden" name="step" value="send_otp">
 <input type="hidden" name="email" value="{{ email }}">

 <button
 type="submit"
 class="btn btn-outline-secondary w-100"
 id="resendBtn"
 disabled
 >
 Resend OTP
 </button>
</form>
{% endif %}

<!-- ===== STEP 3: RESET PASSWORD ===== -->
{% if reset %}
<form method="POST">
 <div class="mb-3">
 <label class="form-label">New Password</label>
 <input type="password" name="password" class="form-control" required>
 </div>

 <div class="mb-3">
 <label class="form-label">Confirm Password</label>
 <input type="password" name="confirm" class="form-control" required>
 </div>

 <button class="btn btn-primary w-100">
 Reset Password
 </button>
</form>
{% endif %}

<div class="mt-3 text-center">
 Back to Login
</div>
```

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

```
 </div>
 </div>
</div>

<!-- ===== OTP TIMER SCRIPT ===== -->
{% if otp_sent %}
<script>
let timeLeft = 300;
const timerEl = document.getElementById("timer");
const resendBtn = document.getElementById("resendBtn");

const countdown = setInterval(() => {
 timeLeft--;
 timerEl.textContent = timeLeft;

 if (timeLeft <= 0) {
 clearInterval(countdown);
 resendBtn.disabled = false;
 resendBtn.textContent = "Resend OTP";
 }
}, 1000);
</script>
{% endif %}

{% endblock %}
```

## 25. db.py

•  
•  
•

# Title: Insuresafe

**Language of Work:** Python, JavaScript,

Framework: Flask,Frontend:

HTML, CSS, JavaScript

Database:MongoDB

.

.

.

.

.

.

.

.

.

.

.